

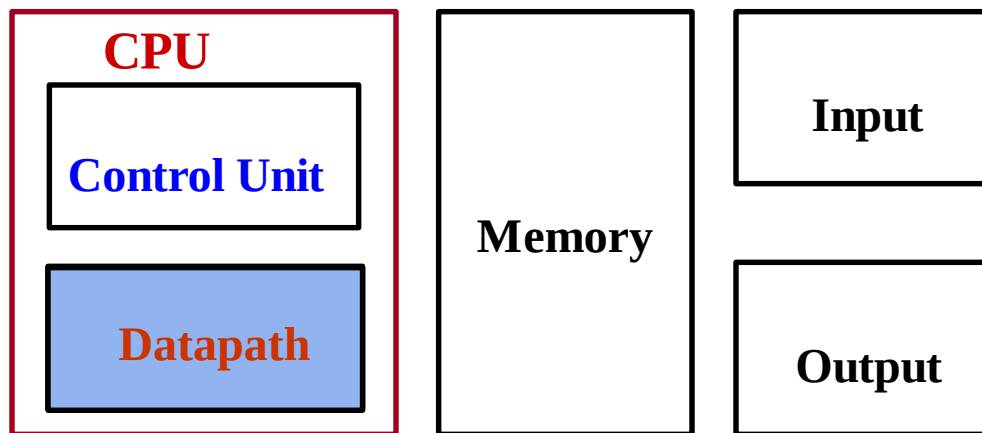
4. CPU原理与设计

主要内容

- 4.1 CPU概述
- 4.2 CPU设计的基本问题
- 4.3 MIPS模型机
- 4.4 单周期处理器设计
- 4.5 流水线技术
- 4.6 多周期处理器设计
- 4.7 流水线冒险
- 4.8 异常处理
- 4.9 指令间的并行
- 4.10 流水线结构实例

4.1 CPU概述

■ CPU的基本结构：



- **数据通路 (DataPath)：**指令的执行部件。指令执行过程中，数据所经过的路径，包括路径中的部件。
- **控制器 (Control Unit)：**指令的控制部件。对指令进行译码，生成指令对应的控制信号，控制数据通路的动作。

CPU的指令执行过程

■ 一条指令的执行过程可以简单分为3个阶段：

1. 提取指令(Fetch)

Instruction Fetch

IF

2. 指令译码(Decode)

Instruction Decode

ID

3. 执行指令(Execute)

IE

第一阶段：提取指令 (FETCH)

取指令 + 自增PC

- 从程序存储器中提取指令，指令的地址由程序计数器（PC）指定，
 - 程序计数器保存当前指令的地址，或者下一条要执行指令的地址
 - 程序计数器是CPU中的一个寄存器
- 提取指令之后，PC会自增，把下一条待执行的指令地址装入PC
 - PC根据指令长度增加存储器单元地址数值

第二阶段：指令译码 (DECODE)

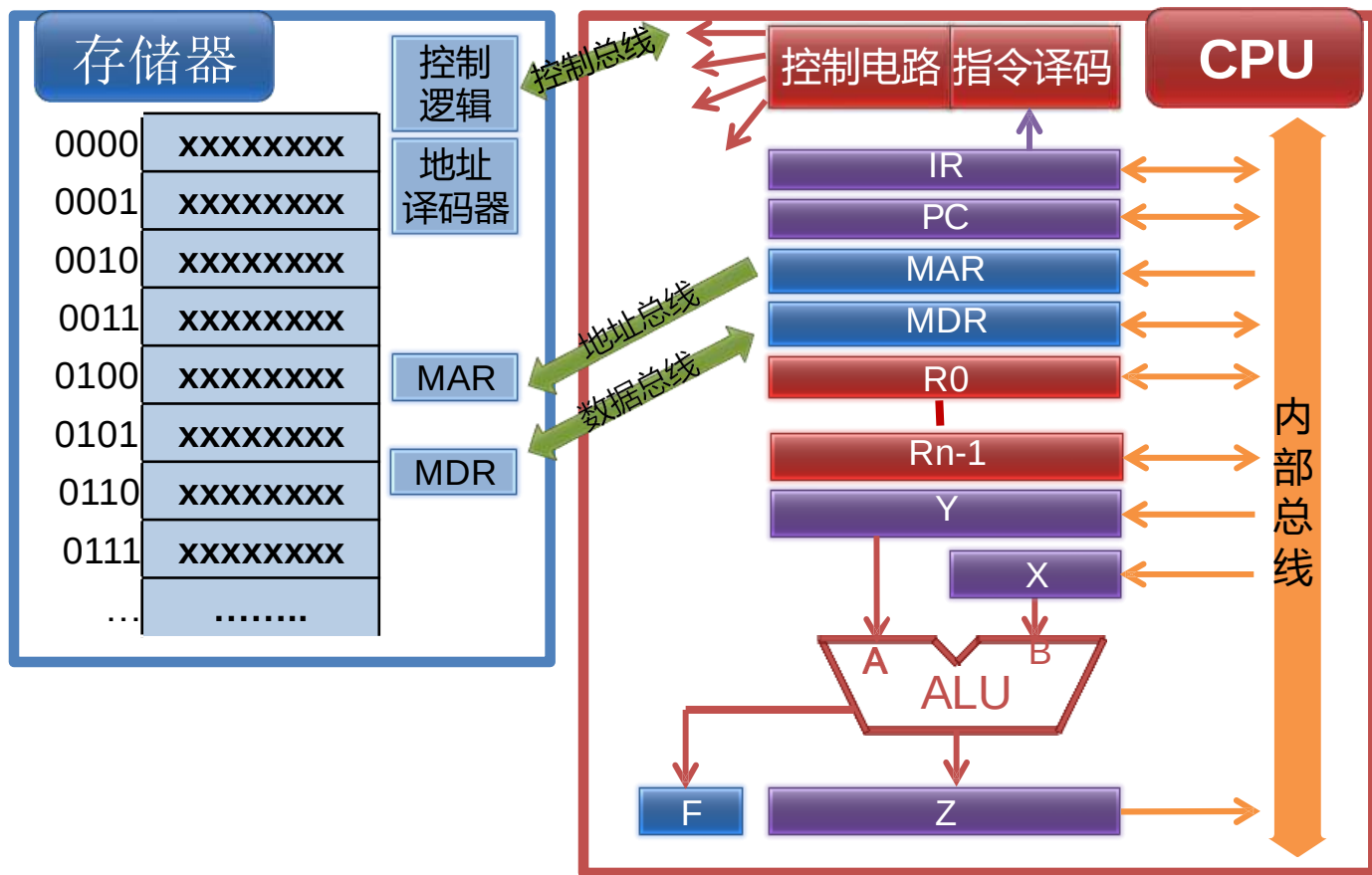
- CPU提取到指令后，在译码阶段，指令分解为操作码和地址码。
 - 操作码：指示CPU要进行哪些运算
 - 地址码：提供运算对象的信息，按照不同的寻址方式访问操作数。
 - 运算对象也许是一个常数值（即立即值），或是一个寄存器的编号（地址），或存储器地址

第三阶段：执行阶段 (EXECUTE)

- 由操作所需的CPU部件协同完成

- 例如，一个加法运算，算术逻辑单元（ALU）会连接到一组输入和一组输出，输入提供了要相加的数值，输出包含加法结果。
- 运算结果的标志可能保存在标志寄存器，如果结果发生溢出，溢出标志被设置。
- 运算结果可能需要保存到寄存器或内存

计算机简化模型



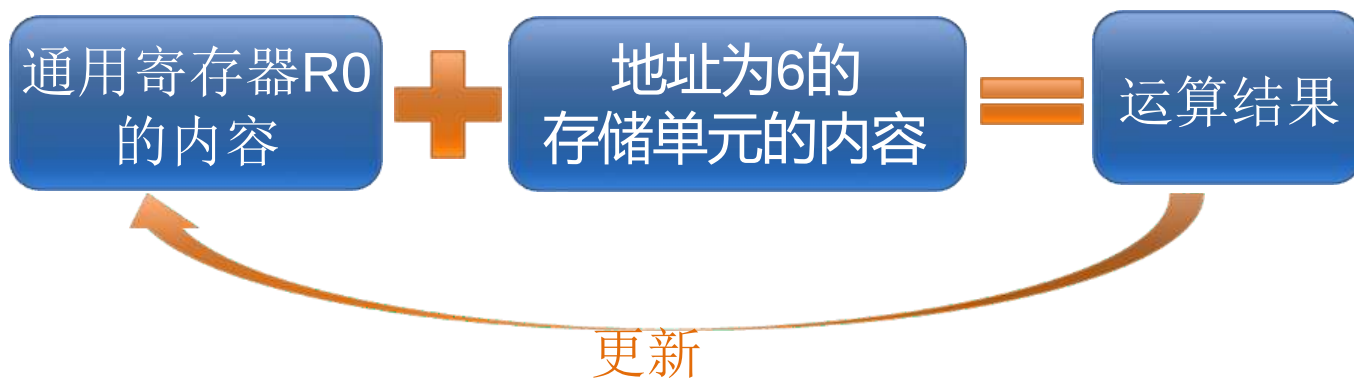
现代CPU中，MAR和MDR不是物理上独立的寄存器，而是集成在内存管理单元（MMU）的功能模块。

内存管理器的地址缓冲区功能上等同于MAR；数据缓冲区功能上等同于MDR。

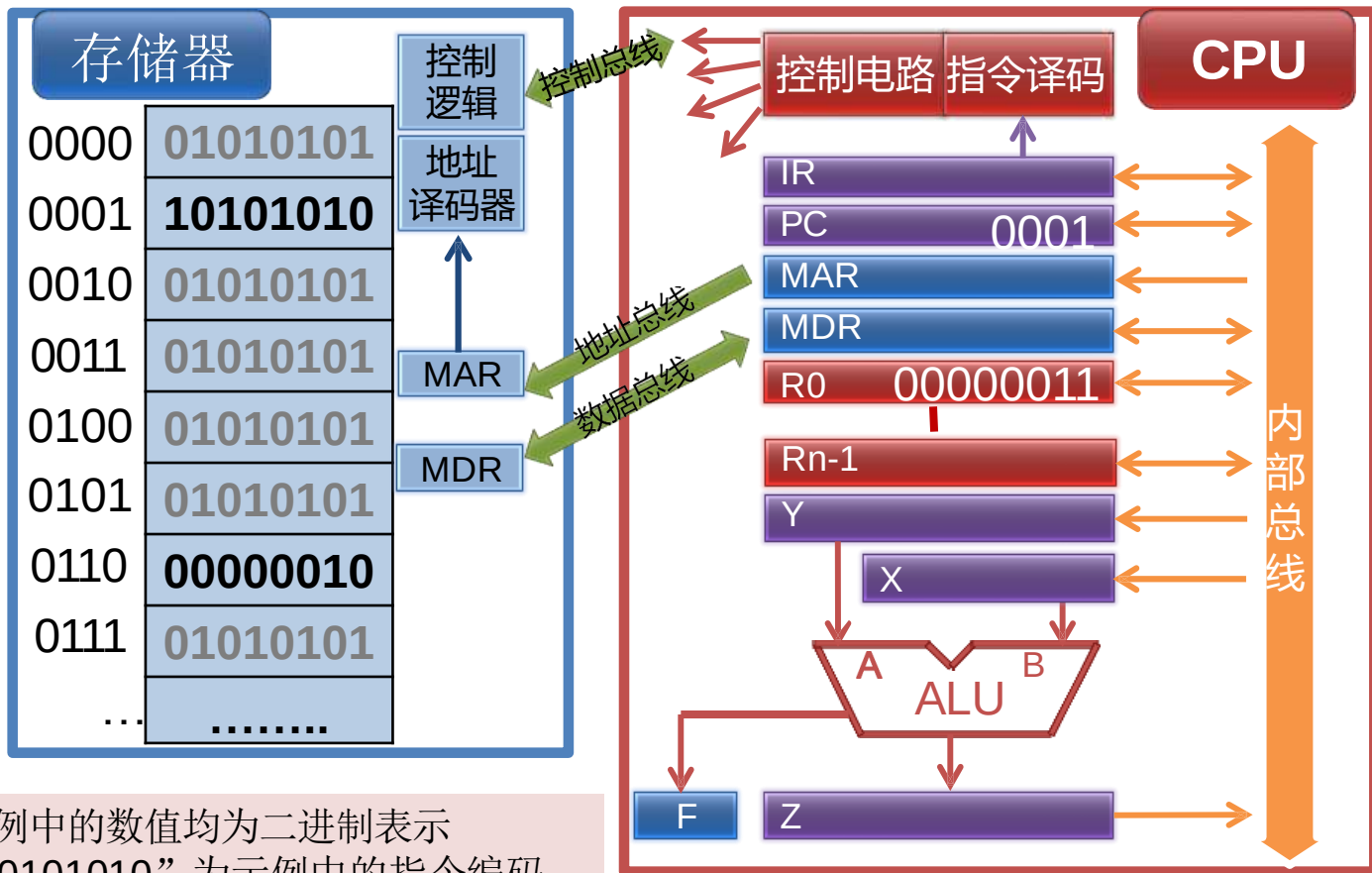
执行指令的示例

指令格式: **ADD R0 , [6]**

指令功能:

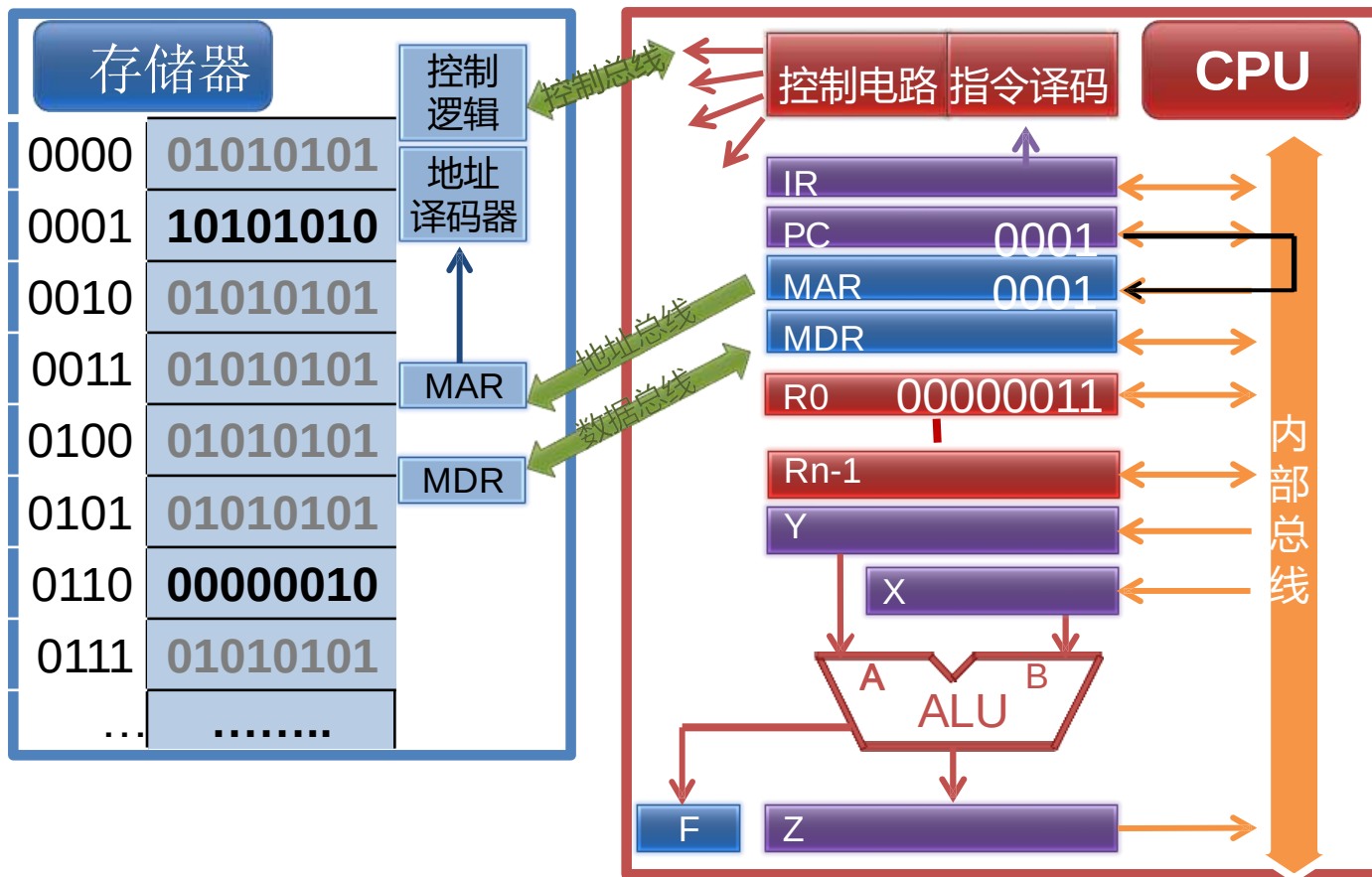


假设的模型机当前状态



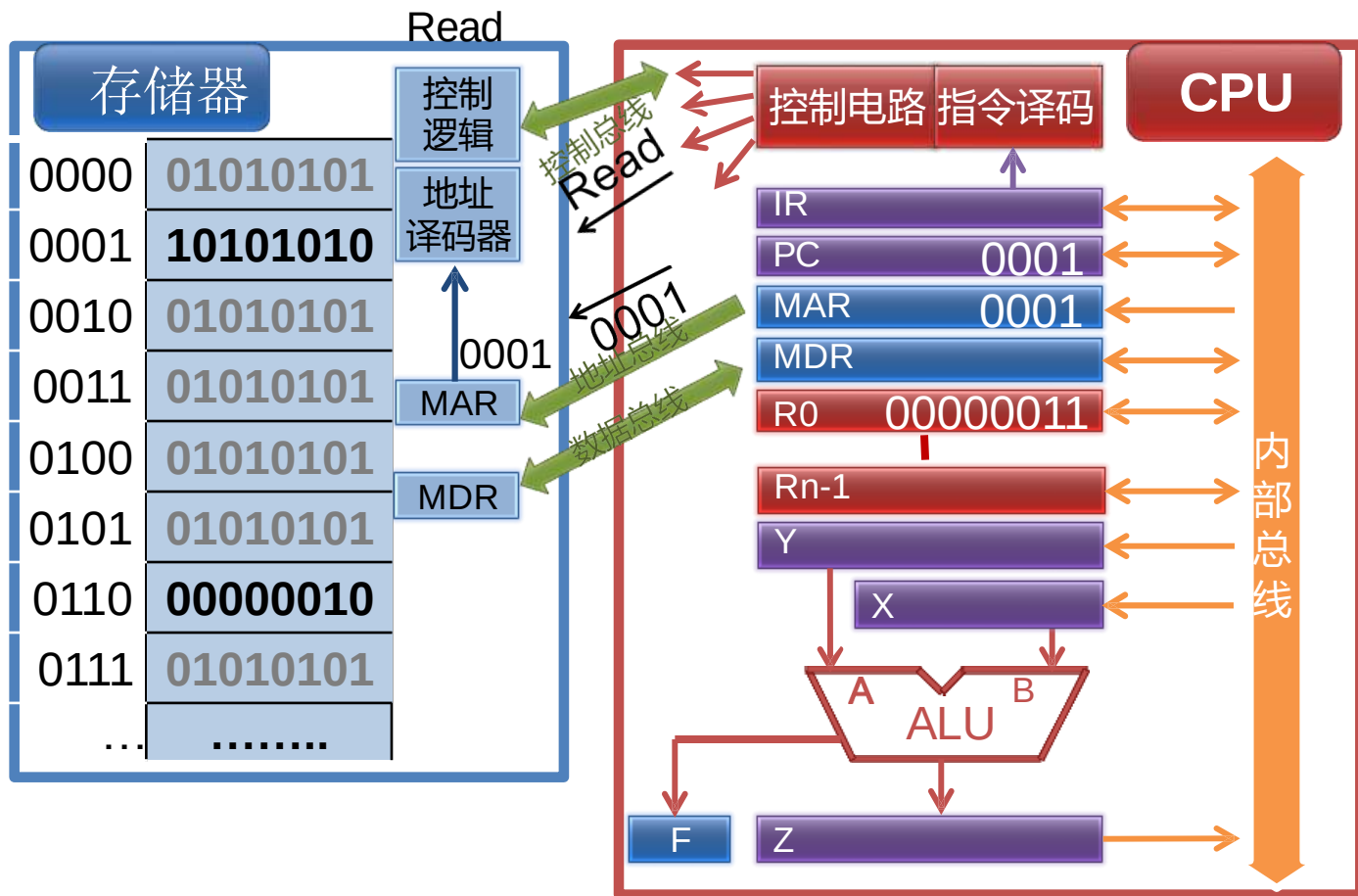
注1: 本例中的数值均为二进制表示
2: “10101010” 为示例中的指令编码

第一阶段：取指（1）

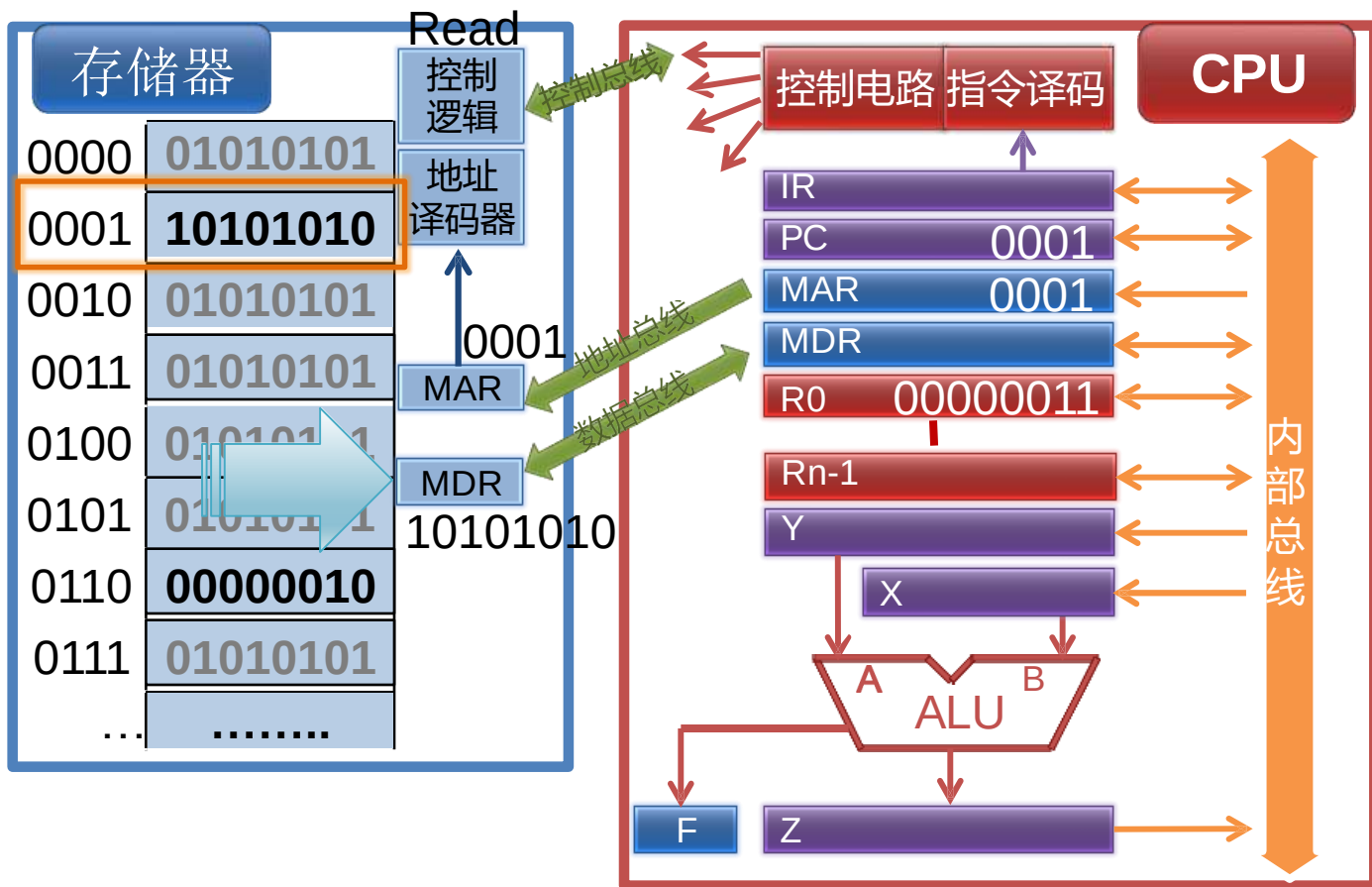


- ① 控制器将指令的地址送往存储器
- ② 存储器按给定的地址读出指令内容，送回控制器

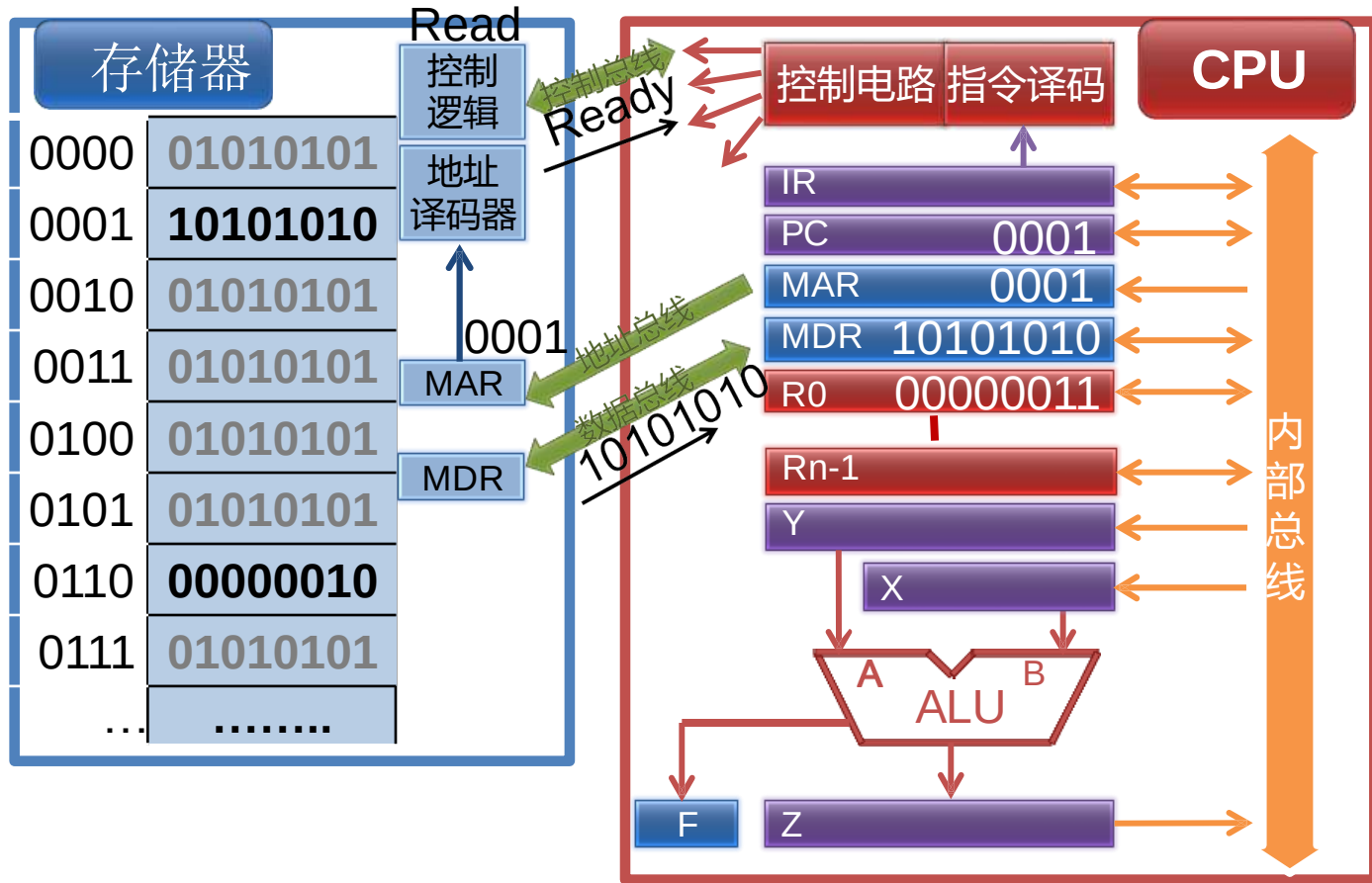
第一阶段：取指 (2)



第一阶段：取指 (3)

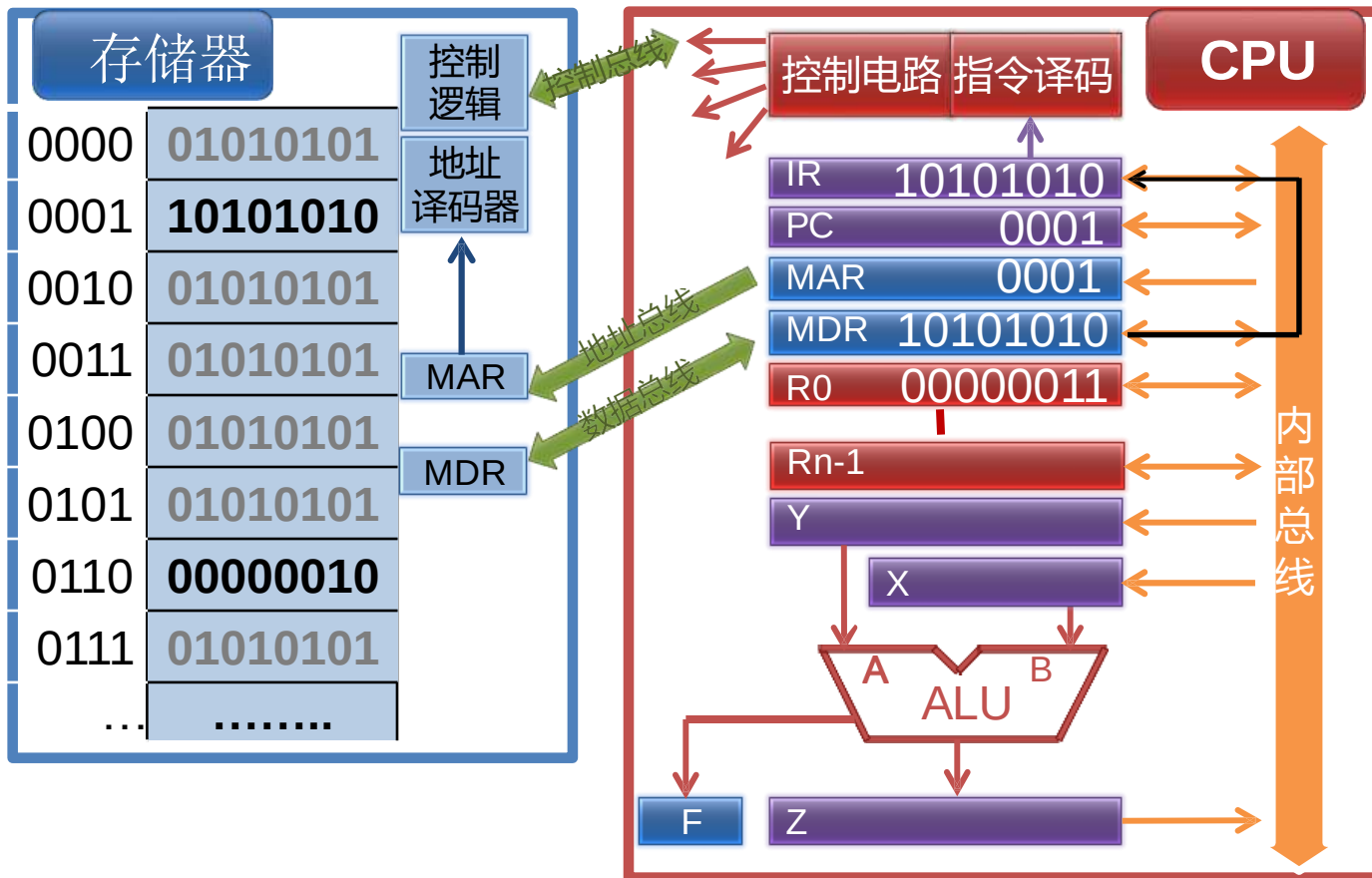


第一阶段：取指 (4)

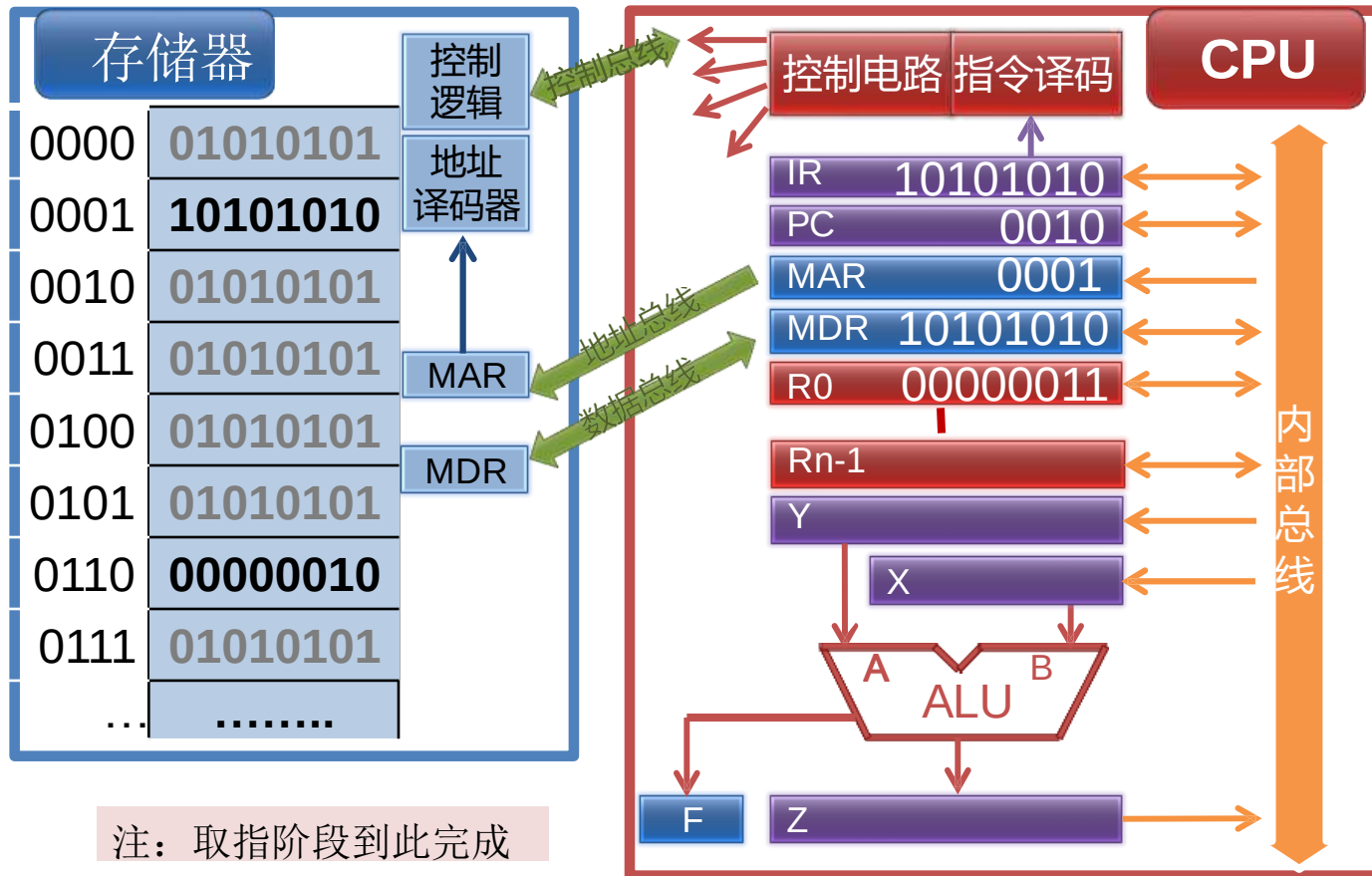


第一阶段：取指 (5)

译码取寄存器内容

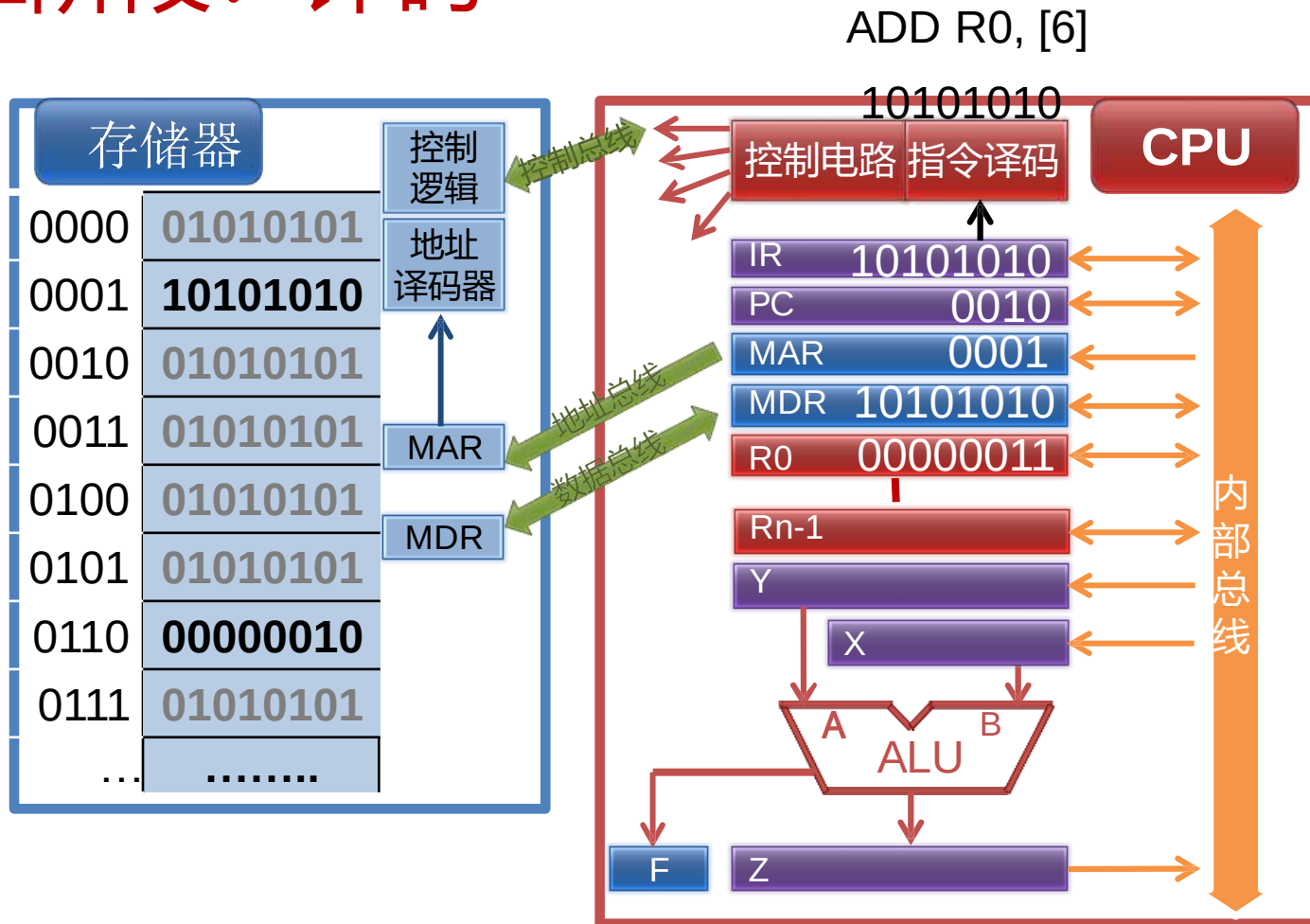


第一阶段：取指 (6)



- PC计数器自增

第二阶段：译码

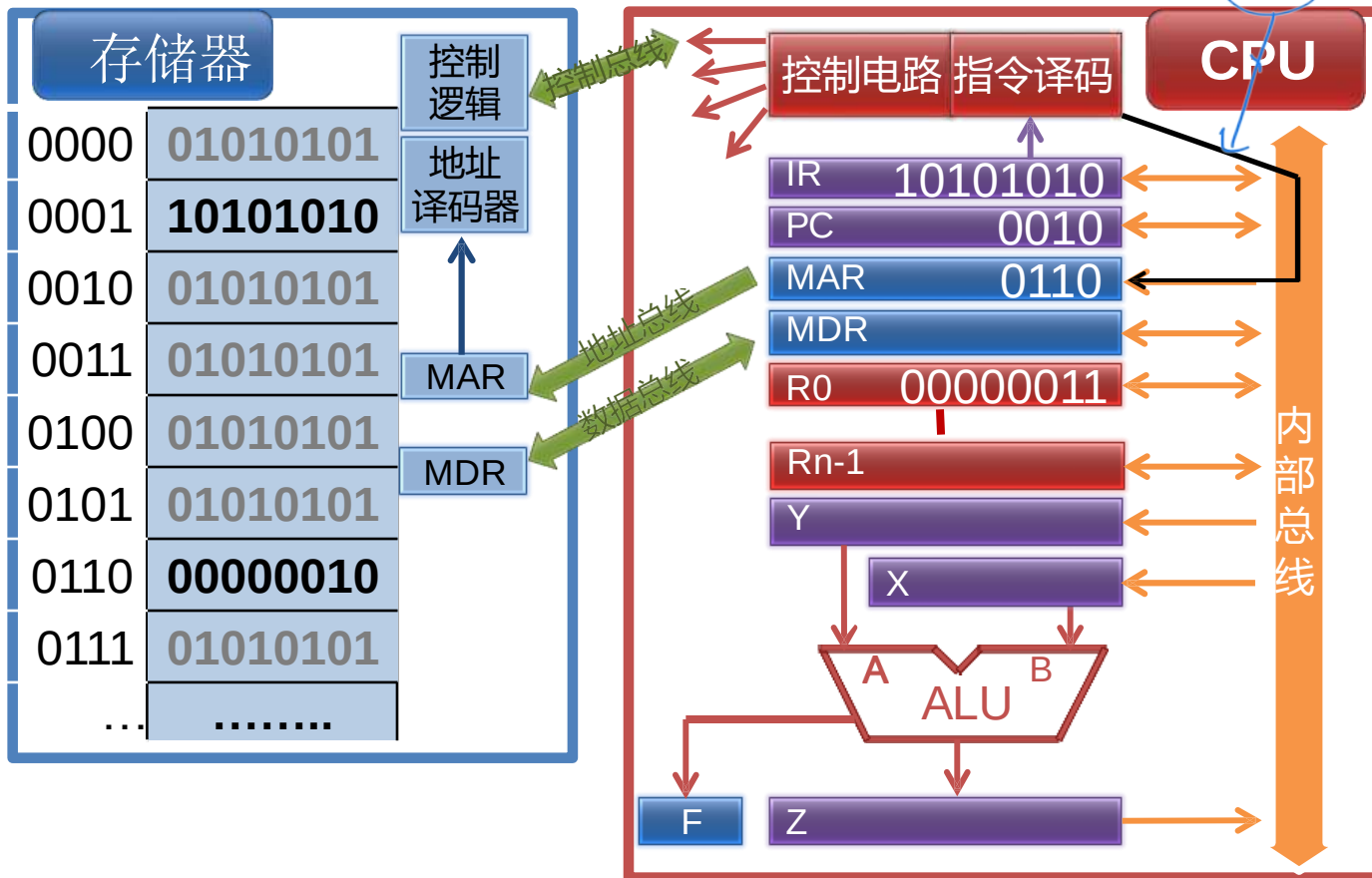


- 控制器分析指令
- 控制器向有关部件发出指令所需的控制信号

第三阶段：执行 (1)

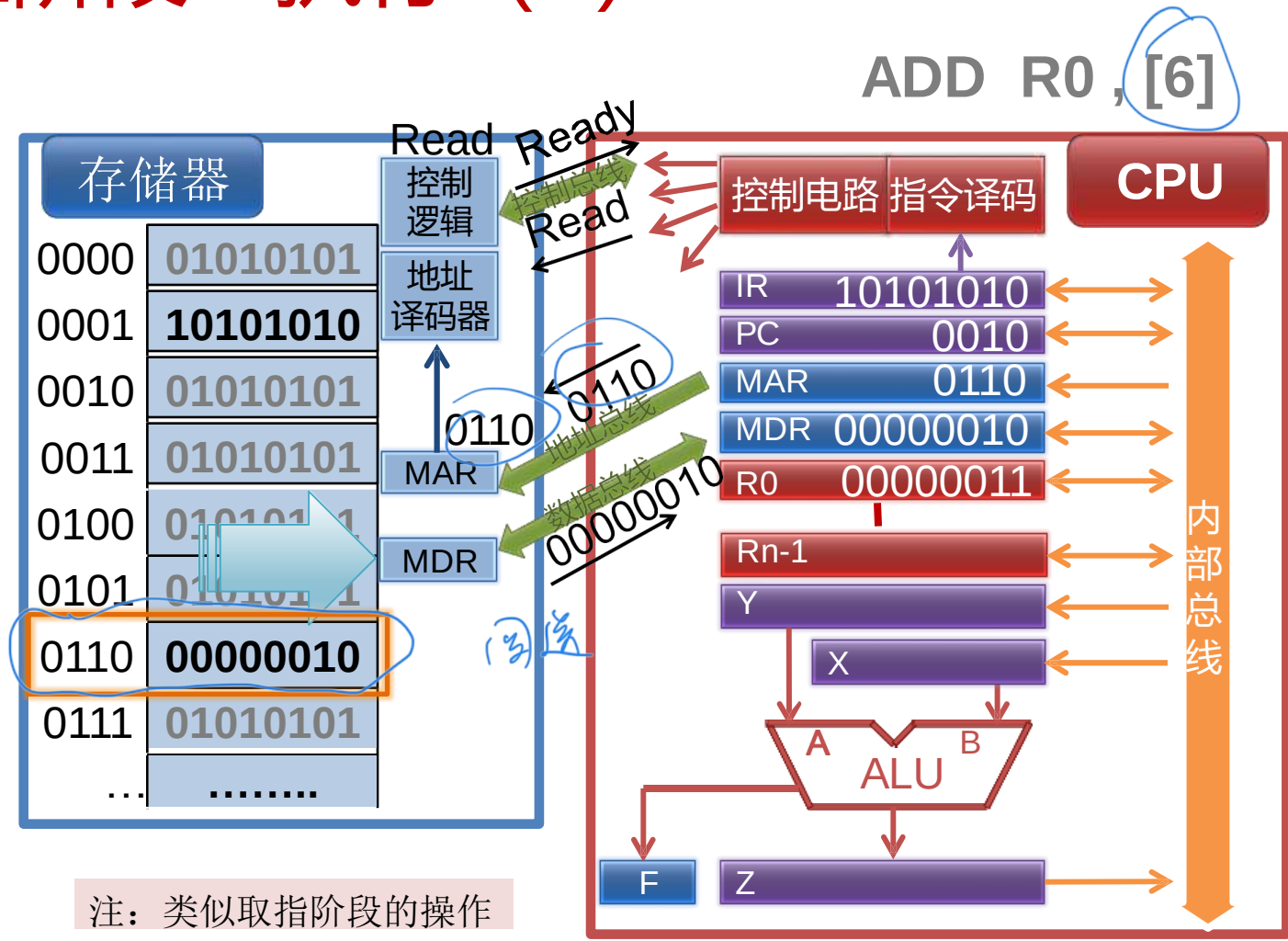
IE

ADD R0, [6]



- 取操作数

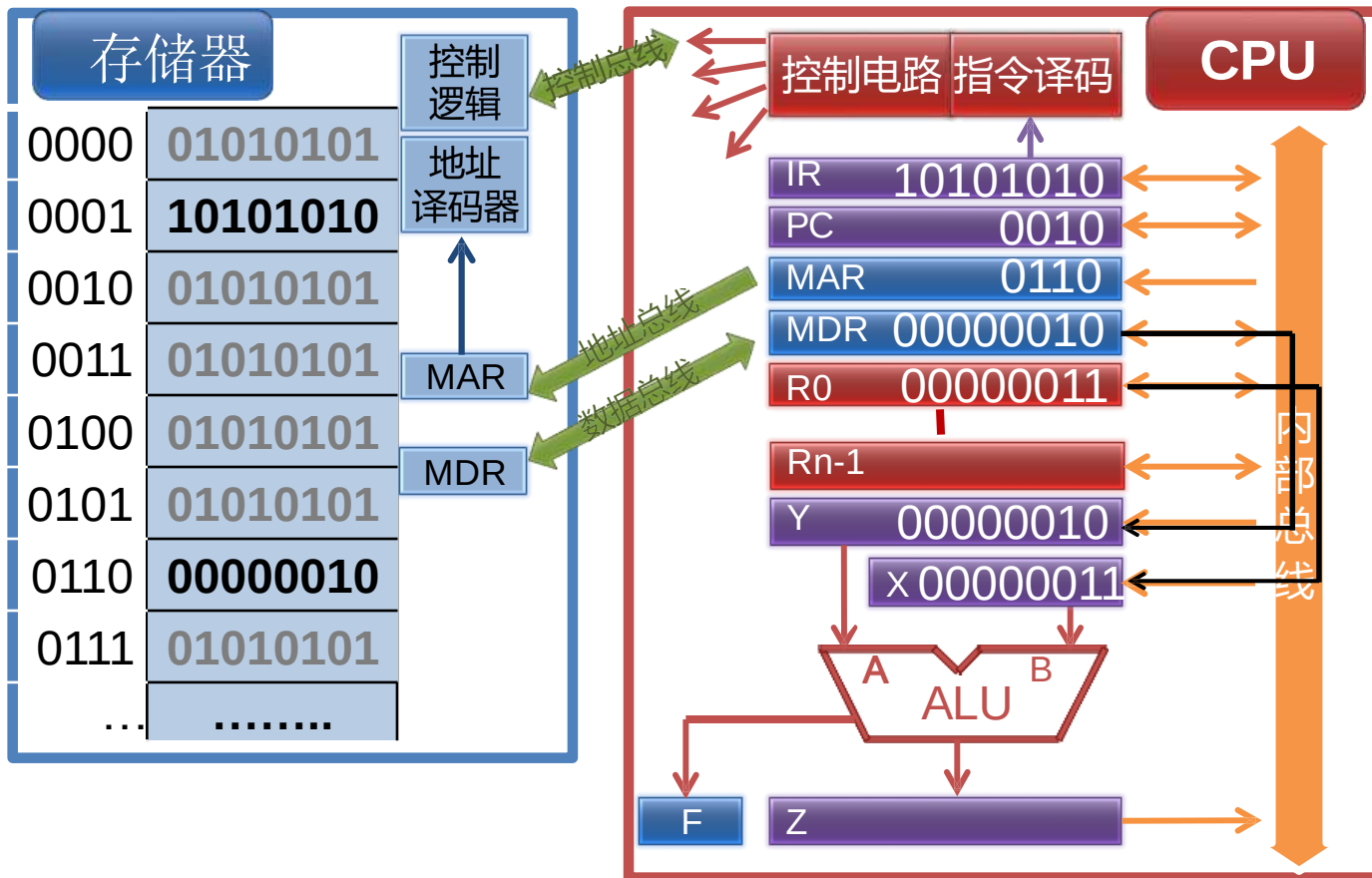
第三阶段：执行 (2)



注：类似取指阶段的操作

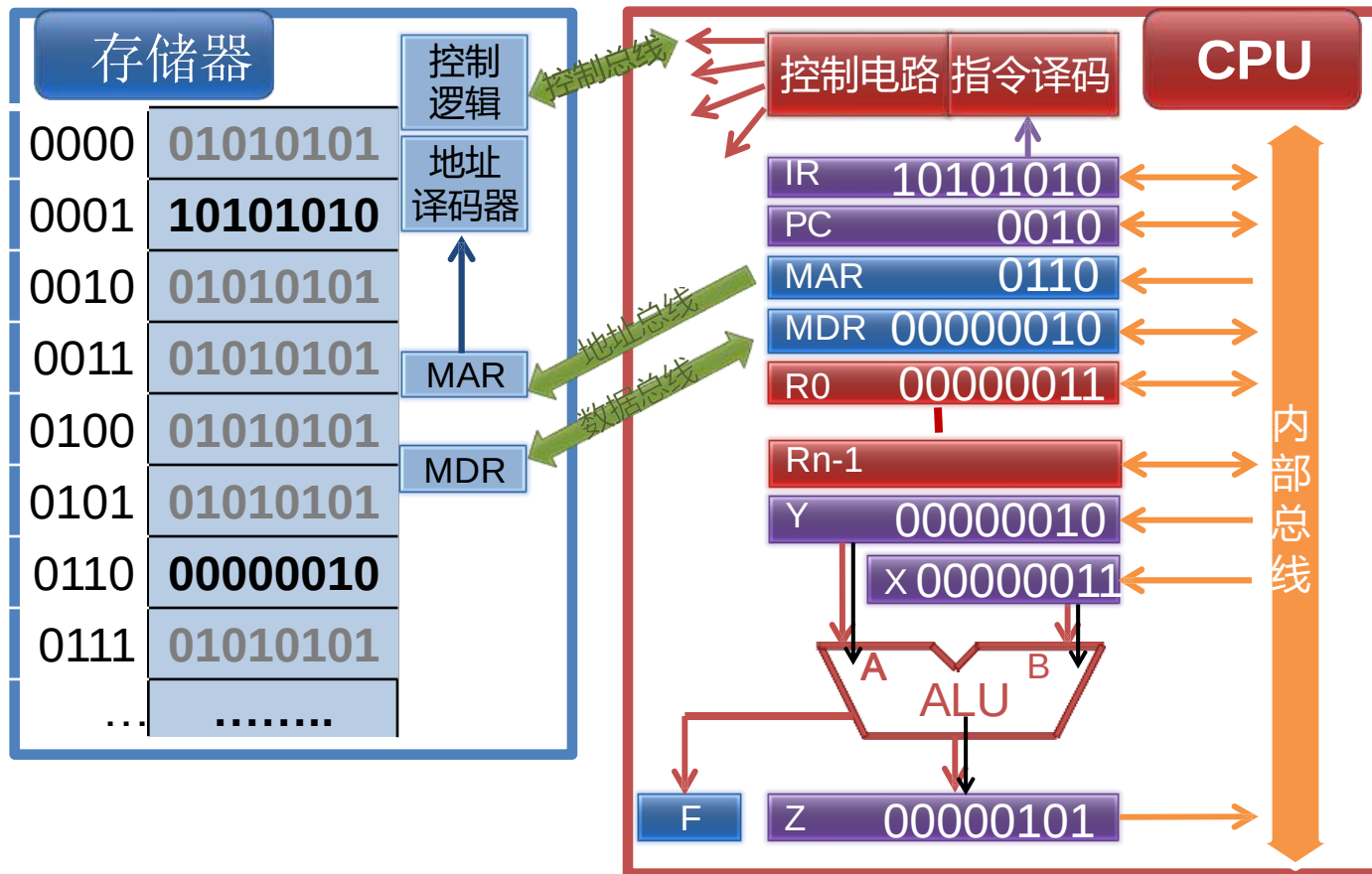
第三阶段：执行 (3)

ADD R0 , [6]



第三阶段：执行（4）

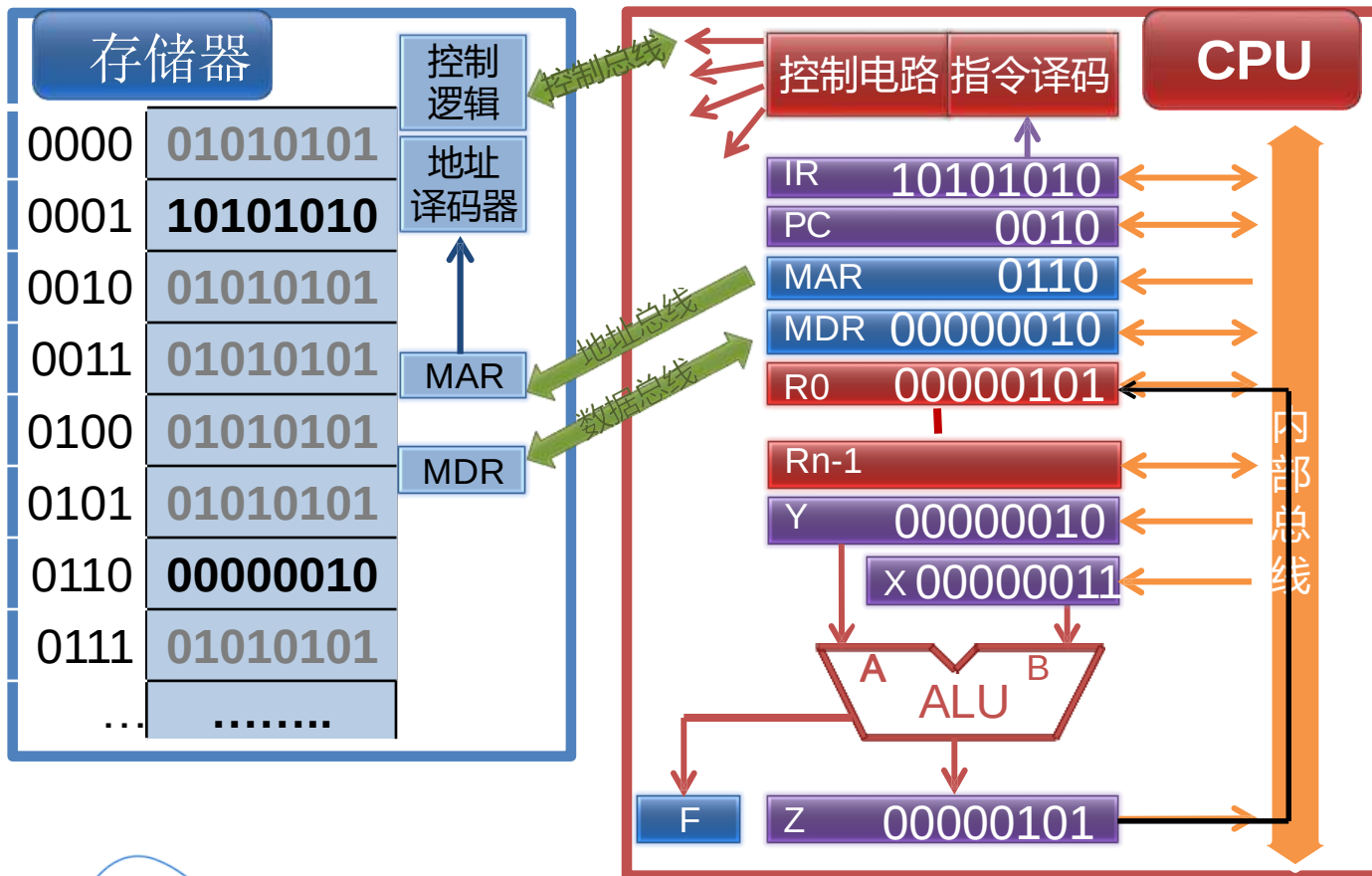
ADD R0 , [6]



- 执行加法计算

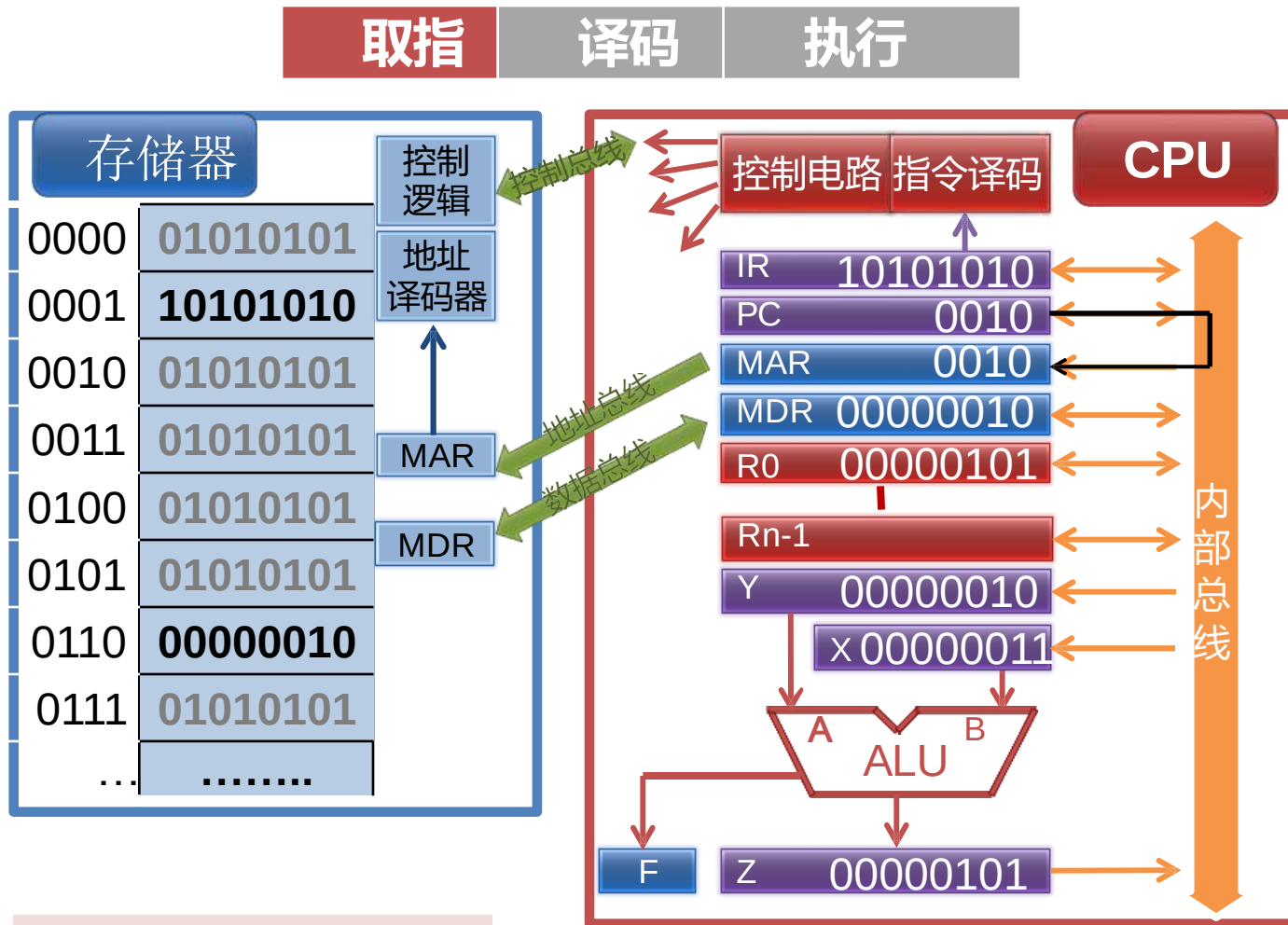
第三阶段：执行 (5)

ADD R0 , [6]



- 结果写回寄存器

下一步



继续执行下一条指令

再次取指

CPU性能与ISA的设计实现

- 比较计算机的性能时，用执行时间来衡量
- CPU 执行时间 = CPU时钟周期数 / 程序 × 时钟周期
= CPU时钟周期数 / 程序 ÷ 时钟频率
= 指令条数 / 程序 × CPI × 时钟周期
- CPI 是以下各方面的综合衡量的结果
 - 程序（算法，编译器）
 - 指令集架构（Instruction Set Architecture, ISA）
 - ISA的实现
 - Organization & Technology

INSTRUCTION SET ARCH. + ORGANIZATION

Computer Design

```
graph TD;
    CD[Computer Design] --> ISD[Instruction Set Design];
    CD --> CHD[Computer Hardware Design];
```

Instruction Set Design

- Machine Language
- "Computer Architecture"
- "Instruction Set Architecture"

"Building Architect"

“建筑设计师”: 功能定义与设计

Computer Hardware Design

- logic & physical Implementation
- "Micro-Architecture"
- "Computer Organization"

"Construction Engineer"

“建造工程师”: 逻辑结构设计和物理实现

例如，是否提供“乘法指令”是ISA设计要考虑的问题；如何实现乘法指令（用专门乘法器还是用一个加法器+移位器实现）是组成(Organization)考虑的问题；如何布线、用什么材料和工艺设计等是实现技术(Technology)考虑的问题。

CPU性能

X表示有影响

■ CPU 执行时间 = 指令条数 / 程序 × CPI × 时钟周期

	指令条数	CPI	时钟周期
算法和编程语言	X	X	
编译器	X	X	
Instr. Set Arch.	X	X	
Organization		X	X
Technology			X

CPU工作的时序

■ 时钟周期

- 由晶体振荡器（其频率叫做时钟频率，主频）确定，周期稳定
- 时钟脉冲是计算机的最小基本工作脉冲，它控制着计算机的工作节奏

■ 机器周期 *（淡化概念）*

- 完成一个基本操作需要的时间，包含一个或多个时钟周期，一般是常量
- 时长设定参考操作时间较长的访问存储器的操作时间 *访存是时间比较长的指令*

■ 指令周期

- 完成一条指令的时间，包含若干个机器周期
- 通常含一个机器周期的指令称为**单周期指令**，包含两个或以上机器周期的指令称为**多周期指令**

主要内容

4.1 CPU概述

4.2 CPU设计的基本问题

4.2 MIPS模型机

4.3 单周期处理器设计

4.4 流水线技术

4.5 多周期处理器设计

4.2 CPU设计的基本问题

- 4.2.1 数据通路的元件
- 4.2.2 简化模型的CPU数据通路设计
- 4.2.3 CPU控制器的实现技术

CPU设计的基本问题

□ 要考虑的几个问题：

① 一条指令由哪些操作组成？

② 一条指令按怎样的顺序完成这些操作？

③ 执行每个操作需要哪些功能单元？

④ 在某个机器周期，功能单元执行哪个操作？

数据通路设计

控制器设计

数据通路：指令执行过程中，数据所经过的路径(包括路径中的部件)，是指令的执行部件。

控制器：控制数据通路的运行，能对指令的执行部件发出控制信号，是指令的控制部件。

CPU的内部结构

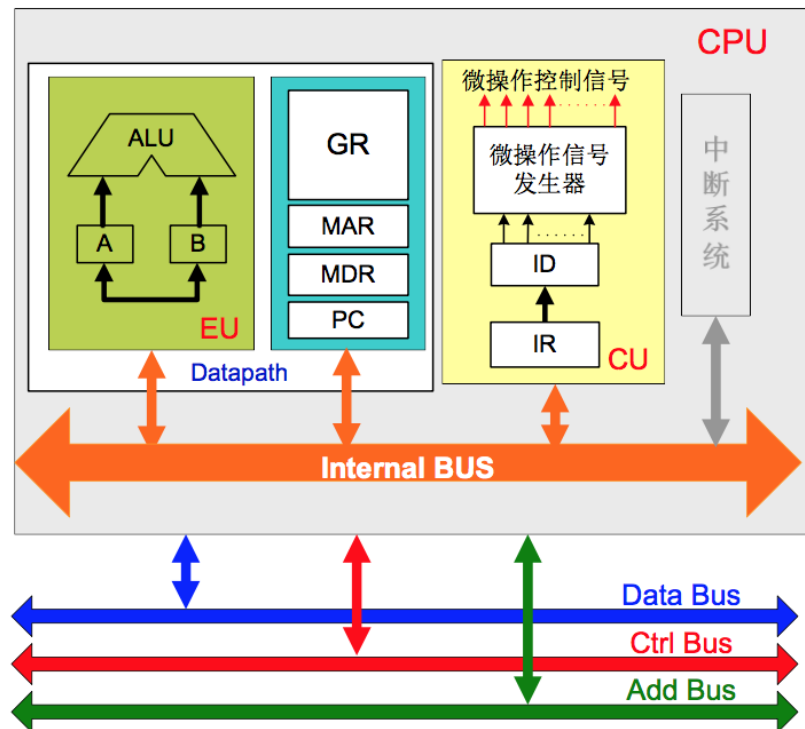
■ 执行单元EU（数据通路，datapath）

- 运算单元：算术逻辑运算单元ALU
- 寄存器：通用寄存器堆GR，主存地址寄存器MAR，主存数据寄存器MDR，标志寄存器（FR，又称程序状态字PSW）等
- 指令地址部件：程序计数器PC

■ 控制单元CU：控制信号产生电路

- 指令寄存部件：指令寄存器 IR
- 译码部件：指令译码器 ID
- 微操作控制信号产生部件：产生计算机其他部件所需要的所有微操作控制信号，有组合逻辑和微程序等实现方式。
- 时序部件：产生时序信号

■ 内部总线



4.2.1 数据通路的元件

- 两类元件：

- 组合逻辑元件（操作元件）

- 时序逻辑元件（状态元件，存储元件）

- 元件间的连接方式

- 总线连接：多个部件共享一组公共的通信线路

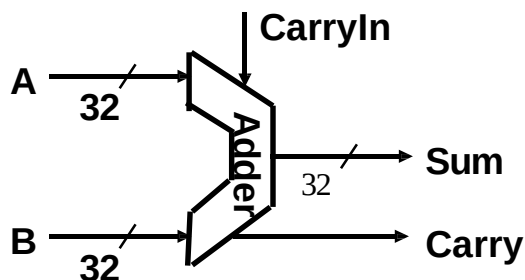
- 分散连接：直接连接

- 数据通路：由“操作元件”和“存储元件”通过总线方式或分散方式连接而成

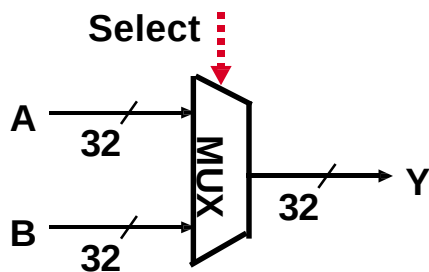
组合逻辑元件

- 组合逻辑元件的特点：
 - 其输出只取决于当前的输入
 - 无需时钟信号：所有输入到达后，经过一定的逻辑门延时，输出端改变，并持续到输入下次改变，不需要时钟信号来定时

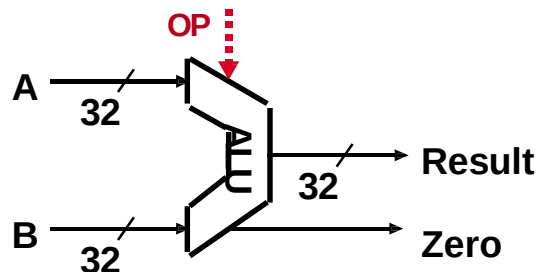
加法器 (Adder)



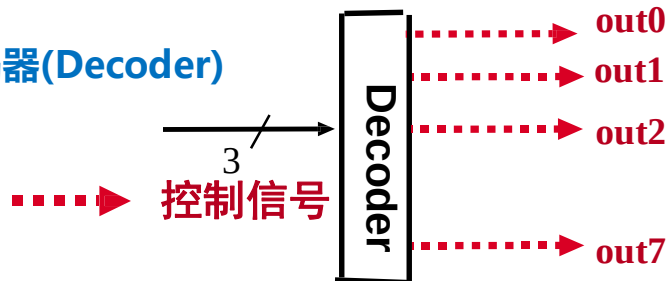
多路选择器 (MUX)



算术逻辑运算单元 (ALU)

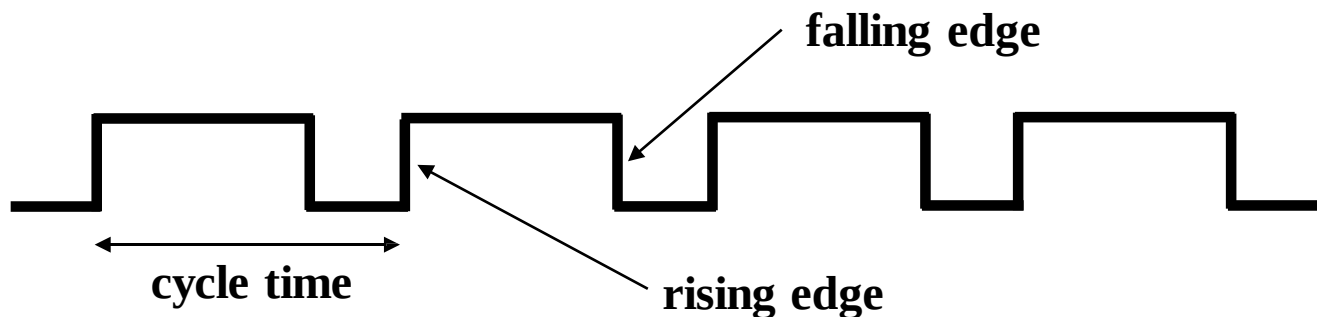


译码器 (Decoder)



时序逻辑元件

- 时序逻辑元件的特点：
 - 具有**存储功能**：在时钟控制下输入被写到电路中，直到下个时钟到达，输入端由时钟决定何时被写入
 - **定时改变**方式：边沿触发（edge-triggered），状态单元中的值只在时钟边沿改变。每个时钟周期改变一次。
 - 上升沿（rising edge）触发
 - 下降沿（falling edge）触发



寄存器与寄存器组

▪ 寄存器 (Register)

PC, IR等

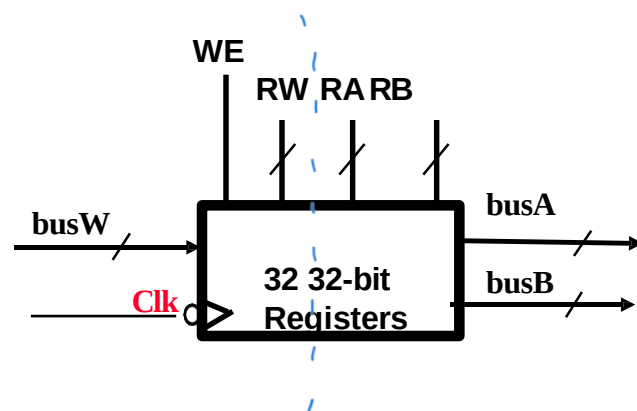
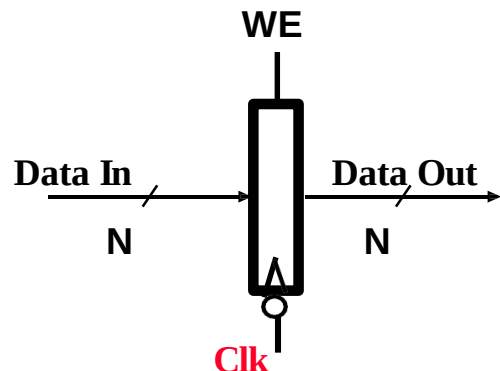
- 暂存信息的逻辑部件，可直接由若干个触发器组成
- 写使能 (WE) 信号有效且时钟沿到来可以进行写操作

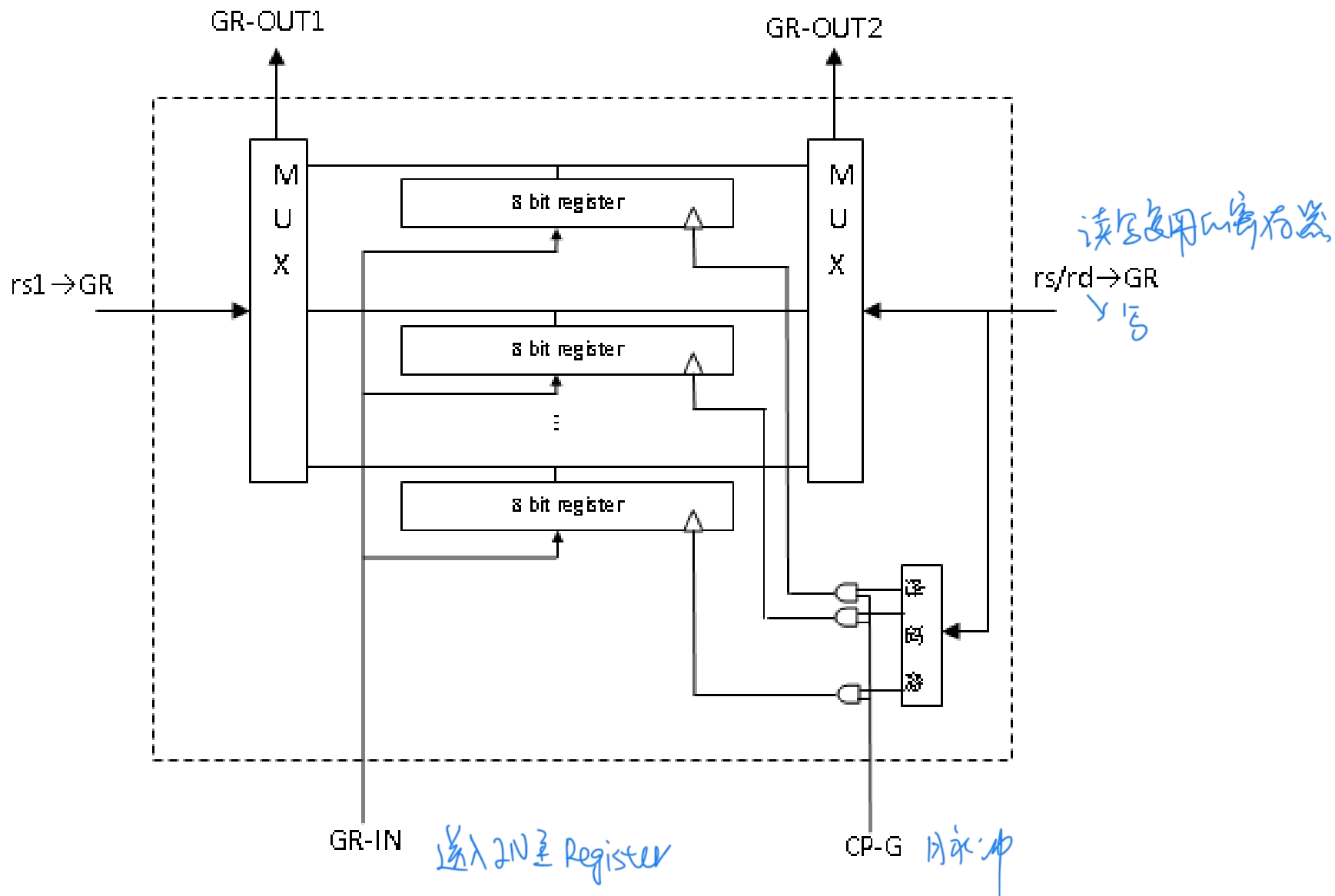
允许写

▪ 寄存器堆 (Register File)

- 由多个寄存器组成，每个寄存器有一个编号，CPU可对指定编号的寄存器进行读写
- 为了提高操作的效率，可以采用双输出总线的结构（主要是为了配合ALU的双输入端）。GR可以一次实现2个寄存器的同时输出，但一次只能实现1个寄存器的输入。

- 两个读口（组合逻辑操作）：busA和busB分别由RA和RB给出地址。
- 一个写口（时序逻辑操作）：写使能有效且时钟边沿到来时，busW传来的值开始被写入RW指定的寄存器中。





通用寄存器组GR内部示意图

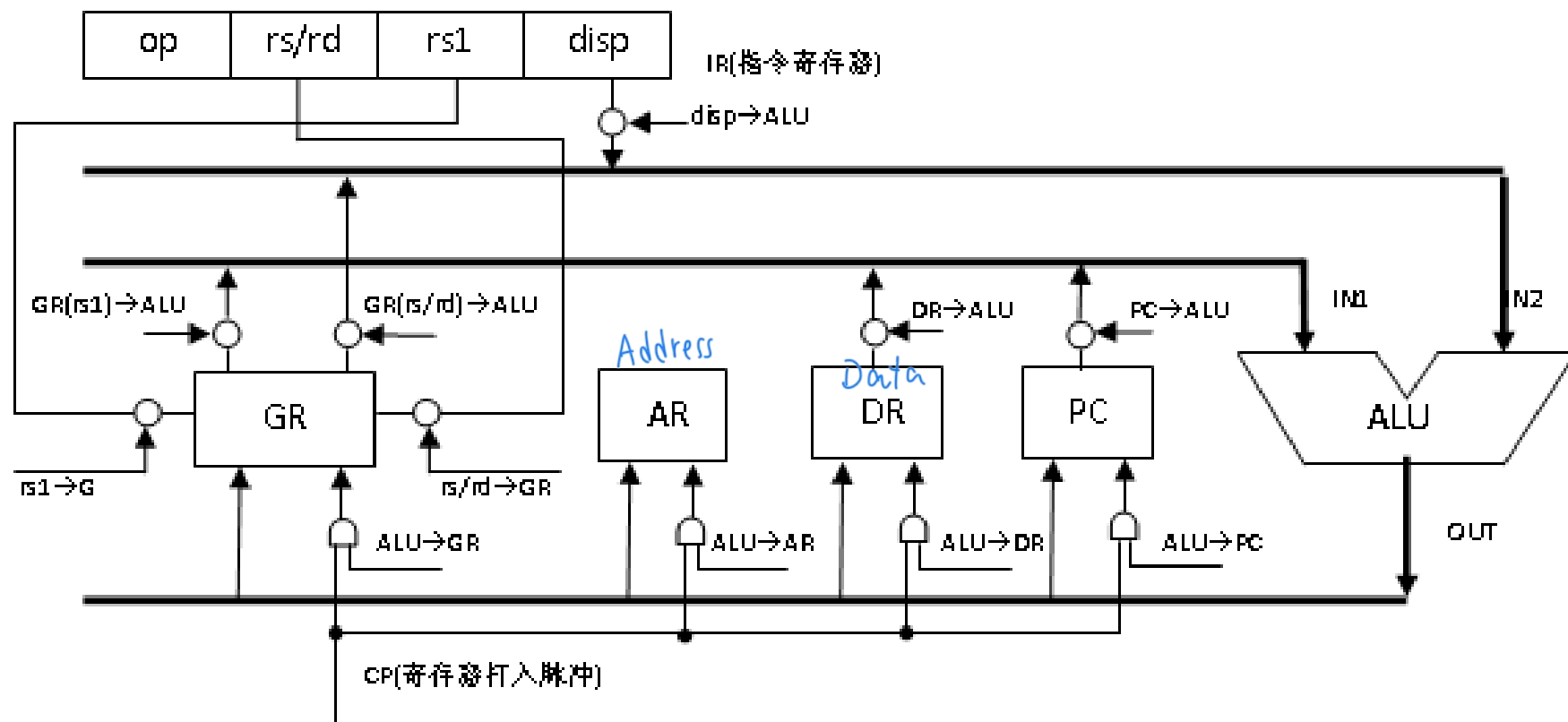
数据通路与时序控制

- 同步系统(Synchronous system)
 - 所有动作有专门时序信号来定时，由时序信号规定何时做什么动作
 - 时序信号：同步系统中用于进行同步控制的定时信号，如时钟信号
 - 例如，指令执行过程每一步都有控制信号控制，由定时信号确定控制信号何时发出、作用时间多长

4.2.2 简化模型的CPU数据通路设计

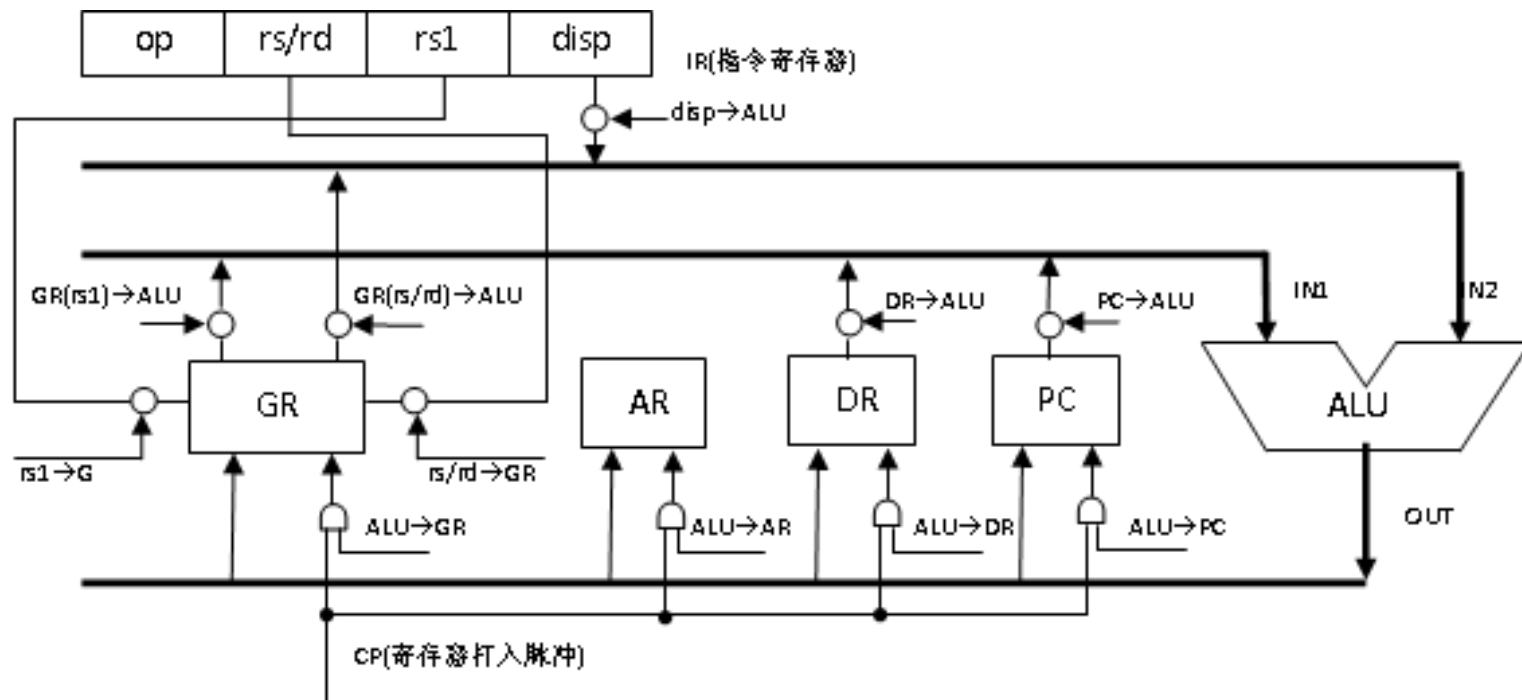
① 为要生成的微命令 (控制信号)

- **CPU内部数据通路**：指令执行过程中，数据所经过的路径(包括路径中的部件)，是指令的执行部件



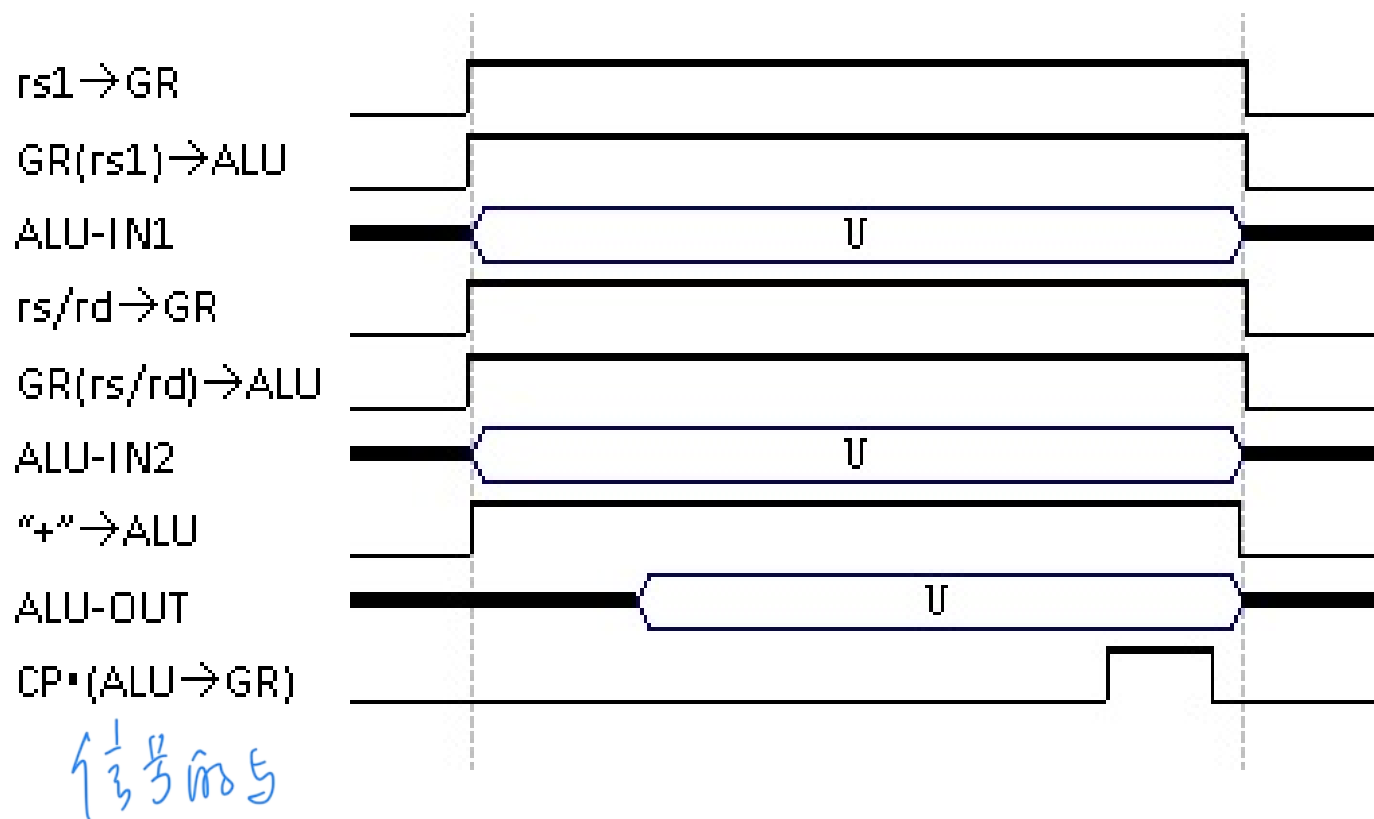
通用寄存器送数据到ALU

- 指定对GR访问指令的寄存器描述字段rs/rd、rs1
 - 指定rs1, 选定寄存器的数据输出到OUT1输出端
 - 指定rs/rd, 选定寄存器的数据输出到OUT2输出端
- 从寄存器中取数到ALU的左输入端, 需要CPU发出控制信号OUT1→ALU-IN1
- 从寄存器中取数到ALU的右输入端, 需要CPU发出控制信号OUT2→ALU-IN2



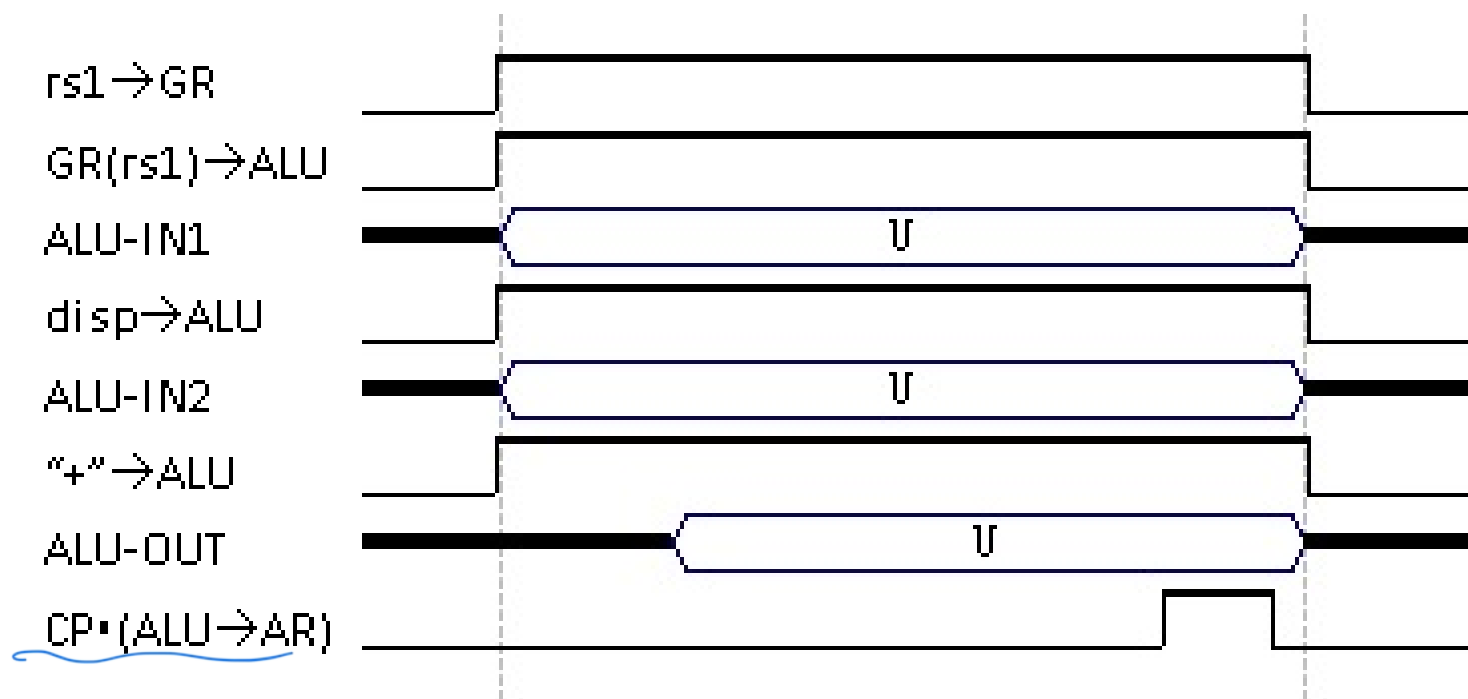
ALU寄存器算术运算时序

两操作数寄存器寻址的控制器信号和数据信号时序



ALU计算地址时序

两操作数立即数寻址的控制器信号和数据信号时序



寄存器需要 CP 地址

单操作数的ALU运算

- 如果是单操作数运算，只发出其中一个控制信号，另一个不发出，默认该操作数为全0。
 - 利用ALU运算将ALU作为传递数据的通路，就是这种用法。

$DR \rightarrow ALU \rightarrow GR$

(从内存中取数放入寄存器，利用ALU，另一个操作数为0)

ALU送数据到通用寄存器

ALU结果输出到总线上，总线上连接了许多部件，只有当某个寄存器的输入脉冲有效时，总线上的数据才能进入该寄存器。

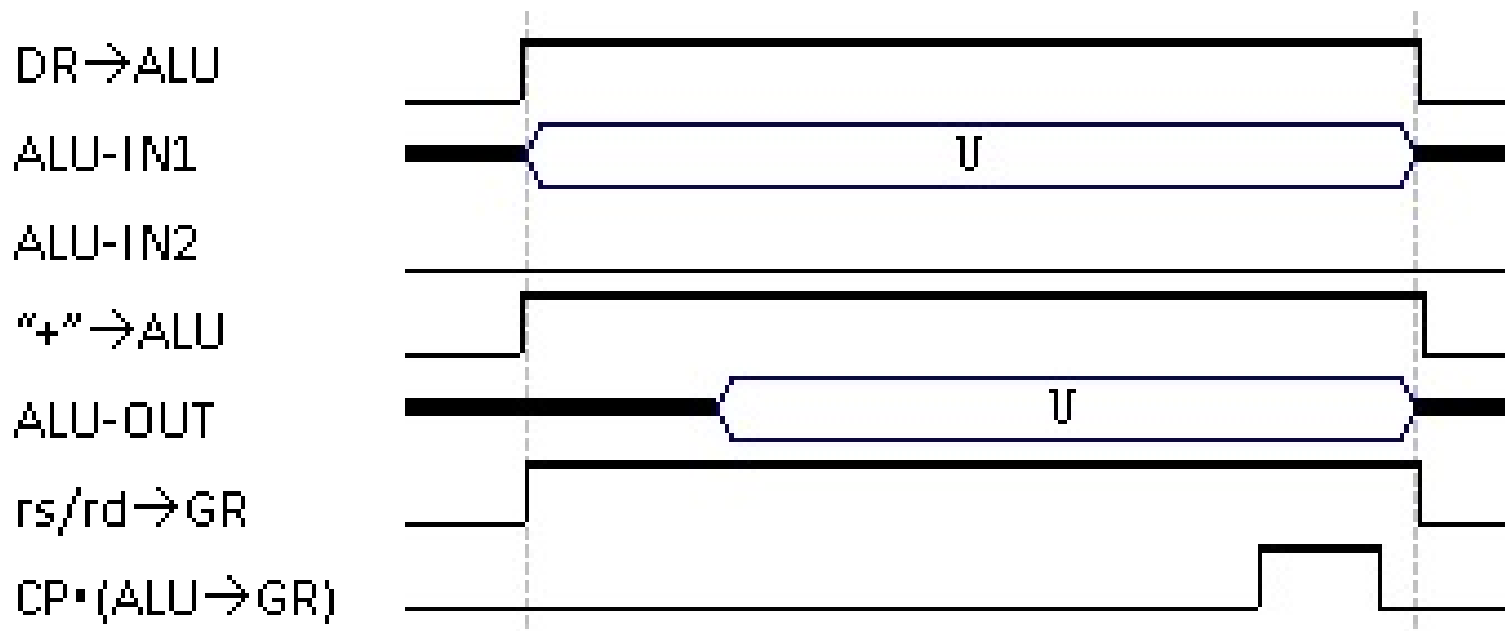
- 按照GR的数据连接方式，CPU发出下列2个控制信号：

rd→GR, ALU→GR

- ALU的输出连接GR的输入总线IN，并且指定输入寄存器的地址为rd
- 再发出一个寄存器的写脉冲（CP），即可将运算结果送入指定的寄存器rd中。

ALU数据送寄存器时序

DR不能直接传送数据到GR，需通过ALU



CPU外部数据通路的控制信号

- CPU的外部部件主要是指**存储器**和**I/O接口寄存器**
 - CPU取指令和读写存储器操作数据时，需要访问存储器
 - CPU执行I/O指令时需要访问I/O接口
- CPU访问外部部件时，CPU提供下列信号：
 - 地址总线AB输出外部部件的**地址信息**，指出CPU要访问的外部部件地址；
例如 MAR
 - **读写控制命令**W/R，W/R=0，表示读操作；W/R=1表示写操作；
 - **存储器和外设接口区分信号**M/IO，M/IO=1，表示访问存储器；M/IO=0，表示访问**外设**。
I/O

CPU访问存储器的两种情况

- CPU取指令操作，此时只有存储器读操作，没有写操作。
 - 取指令阶段，CPU将程序计数器PC的内容送地址总线AB
 - 执行存储器读操作， $W/R=0$ ， $M/IO=1$ ，存储器将读取的结果送数据总线DB，CPU将数据总线上的数据（此时是指令）送指令寄存器IR。
- CPU读写数据操作，此时既有读操作，也有写操作。
 - CPU将数据地址寄存器AR的内容送地址总线AB，根据读或写，执行以下操作：
 - 存储器读操作： $W/R=0$ ， $M/IO=1$ ，存储器将数据取出放在数据总线DB上，CPU将数据总线DB上的数据写入CPU的数据寄存器DR中。
 - 存储器写操作：CPU将数据寄存器DR的内容输出到数据总线DB上，置 $W/R=1$ ， $M/IO=1$ ，数据将写入存储器。

存储器读操作时序

地址送地址总线

AR → AB

AB

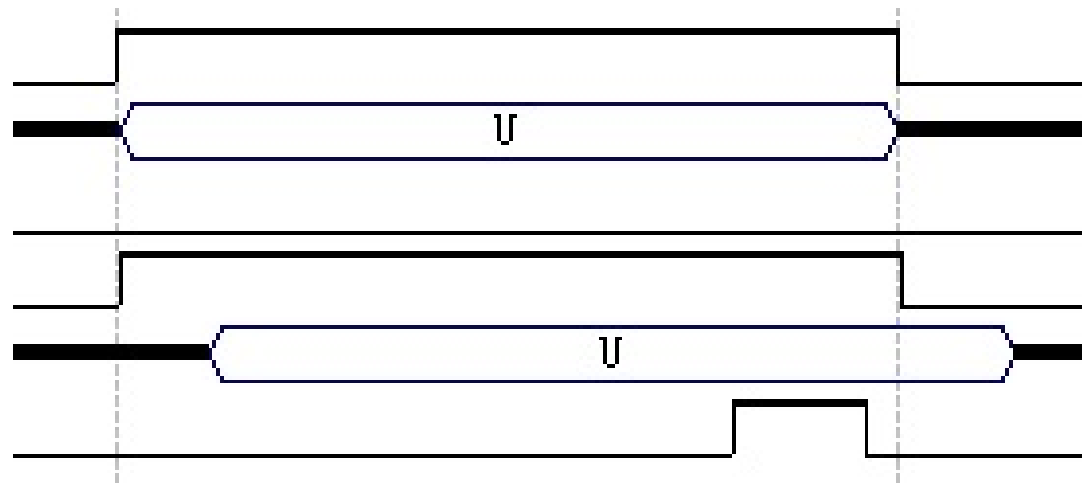
W/R=0

M/I O=1

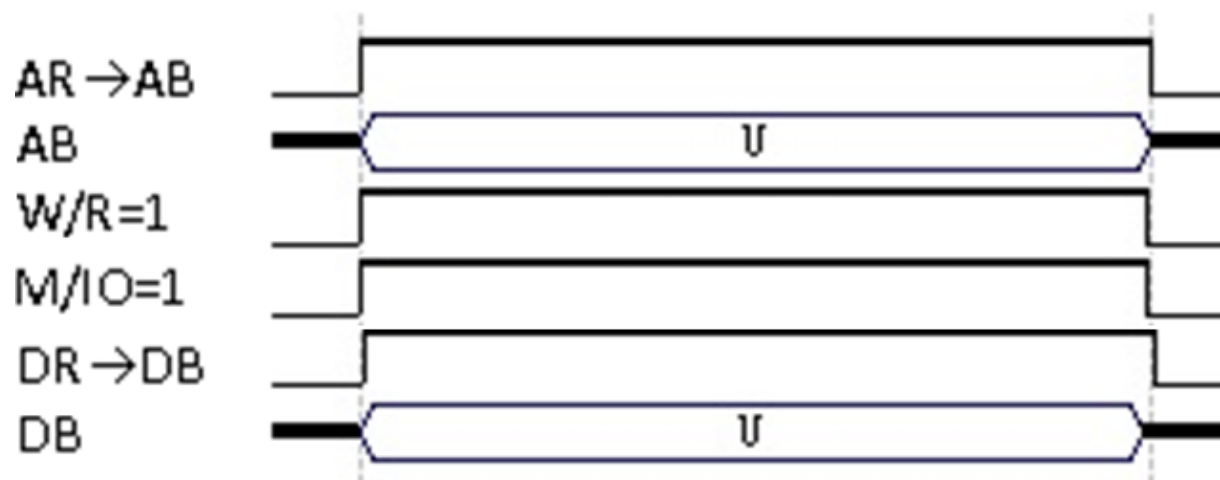
DB

CP • (DB → DR)

寄存器



存储器写操作时序



控制器

- 控制单元 (Control Unit, CU) :
 - 生成数据通路各元件的控制信号
 - 例如：ALU做什么运算；GR读取哪两个寄存器、写入哪个寄存器
 - 控制数据通路：通过多路选择器控制数据的流向
 - 根据不同指令、不同阶段给出控制信号
 - 控制信号繁多，是CPU中最为复杂的电路

基本指令的数据通路

- CPU内各个部件之间的关系和数据流

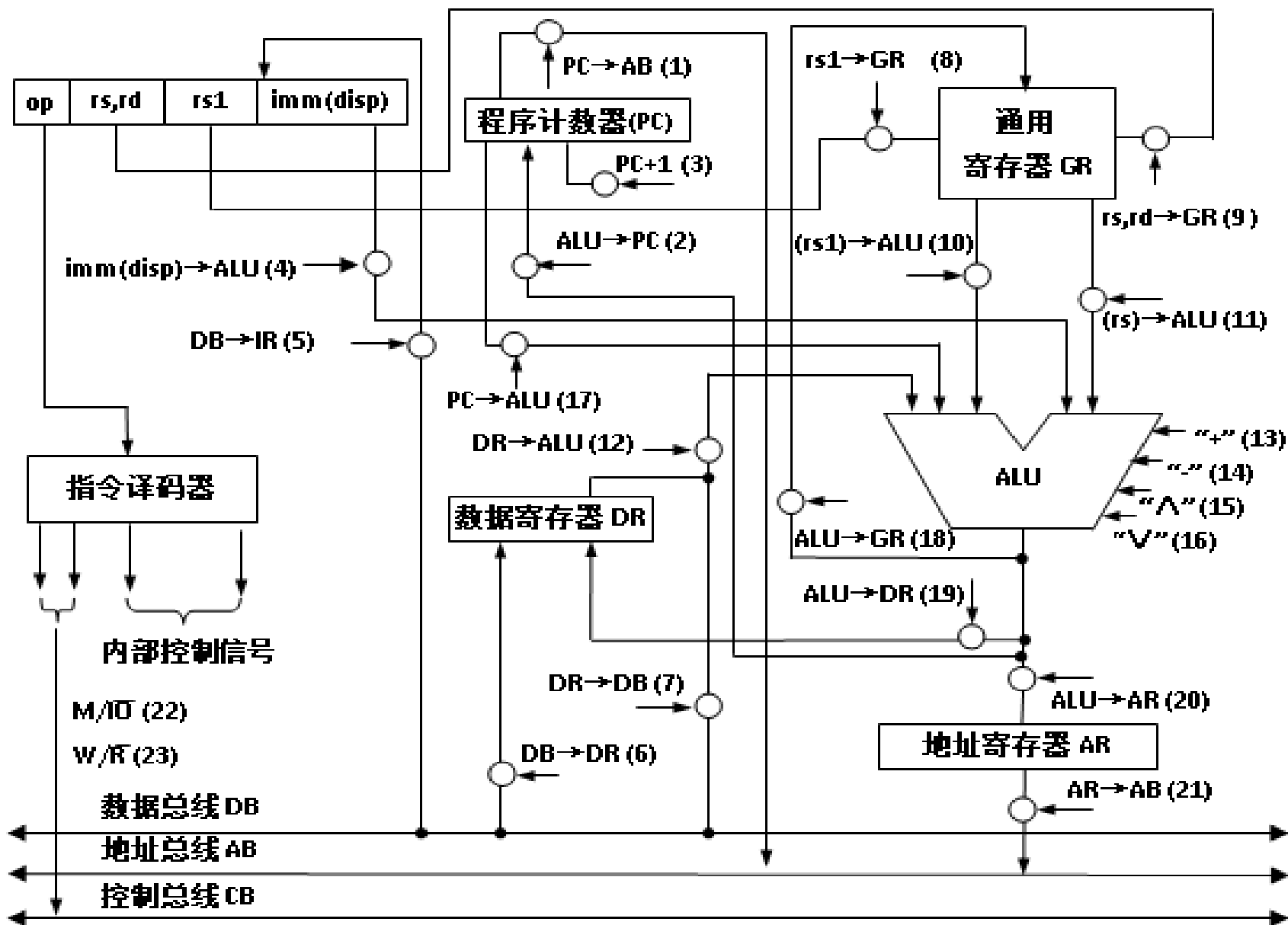
- CPU中数据通路的完整描述含CPU内部数据通路和外部数据通路

- 假设：

- 采用定长指令，所有的指令长度相等，取指令的时间是一样的；
 - 取指令只占用一个机器周期

- 指令的执行可能包含的机器周期有多种情况：

- 1个机器周期情况：如空操作、停机、条件转移不成功时等，CPU只需要一个机器周期；
 - 2个机器周期情况：如算术逻辑指令、传送指令、移位指令等；
 - 4个机器周期情况：主要是Load/Store指令。



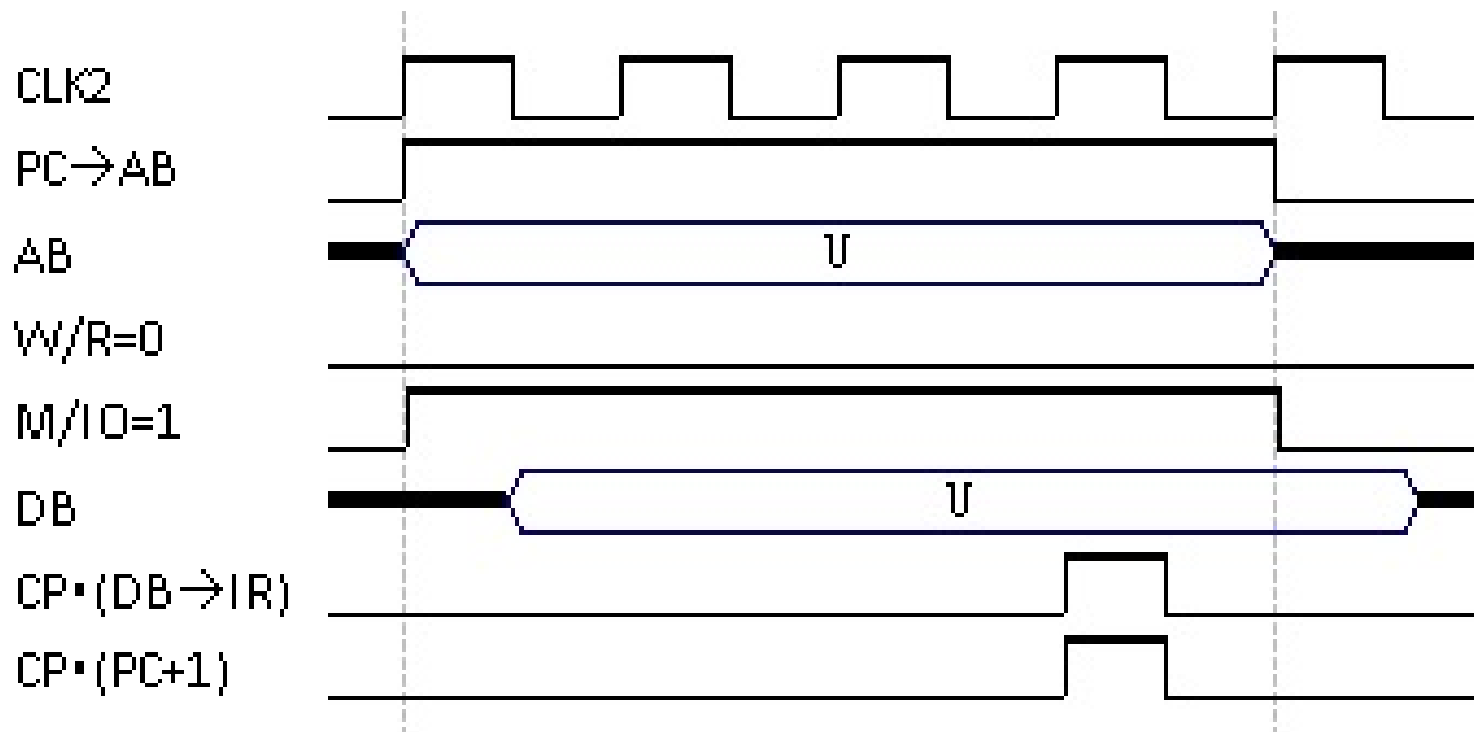
长实线：CPU内各个部件之间的数据流通路
小圆圈“o”：控制该数据通路的控制信号

公共机器周期：取值周期

取指周期CPU发出的控制信号如下：

- $PC \rightarrow AB$ ，将PC计数器的内容送地址总线AB
- $W/R=0$ ，发读控制信号给存储器
- $M/IO=1$ ，表示访问存储器
- $CP \bullet (DB \rightarrow IR)$ ，将存储器读出的数据送IR，表示控制信号 $DB \rightarrow IR$ 和打入脉冲CP逻辑与。
- $CP \bullet (PC+1)$ ，为了读取下一条指令地址增量计算。

公共机器周期的时序图



在一个机器周期最后三个时钟脉冲中完成一个写操作

1) LOAD指令分析

- Load指令: Load rd, ((rs1)+disp)
 - 表示将存储单元((rs1)+disp)的内容（操作数）送到寄存器rd中。
 - 执行过程包含计算存储单元的地址，读存储单元的数据，数据送寄存器rd。
- 数据送寄存器rd需要借用ALU运算，利用ALU通路
 - 数据通路中没有将DB内容直接送到通用寄存器的通路，DB中的数据可以送到DR，但DR也没有直接到通用寄存器的通路
 - DR必须经过ALU才能到达通用寄存器，即完成一次ALU运算，其中一个操作数的输入是DR，另一个操作数为空，操作结果放寄存器rd。

LOAD指令的分析

load指令一共需要4个机器周期：

1. 取指：操作内容同公共机器周期。

2. 计算地址：实现存储单元的地址计算，主要通过加法操作。

- ① 一个操作数是rs1, 另一个操作数是disp, 可以直接从指令寄存器IR获得
- ② 将这两个操作数分别送入ALU的左右两个输入端。
- ③ 向ALU发出 “+” 操作信号, 在ALU的输出端得到计算结果。
- ④ 生成送数信号脉冲 $CP \cdot (ALU \rightarrow AR)$, 实现将运算结果送入AR, 完成地址的计算。

CPU发出的控制信号如下：

- $rs1 \rightarrow GR$, 寄存器地址rs1送GR
- $GR(rs1) \rightarrow ALU$, 寄存器rs1的数据输出到ALU
- $disp \rightarrow ALU$, 地址偏移量送ALU
- “+” $\rightarrow ALU$, ALU选择加法操作
- $CP \cdot (ALU \rightarrow AR)$, ALU计算结果送寄存器AR

基址

偏移

LOAD指令的分析

3. 读取操作数：完成从存储器中读取操作数。

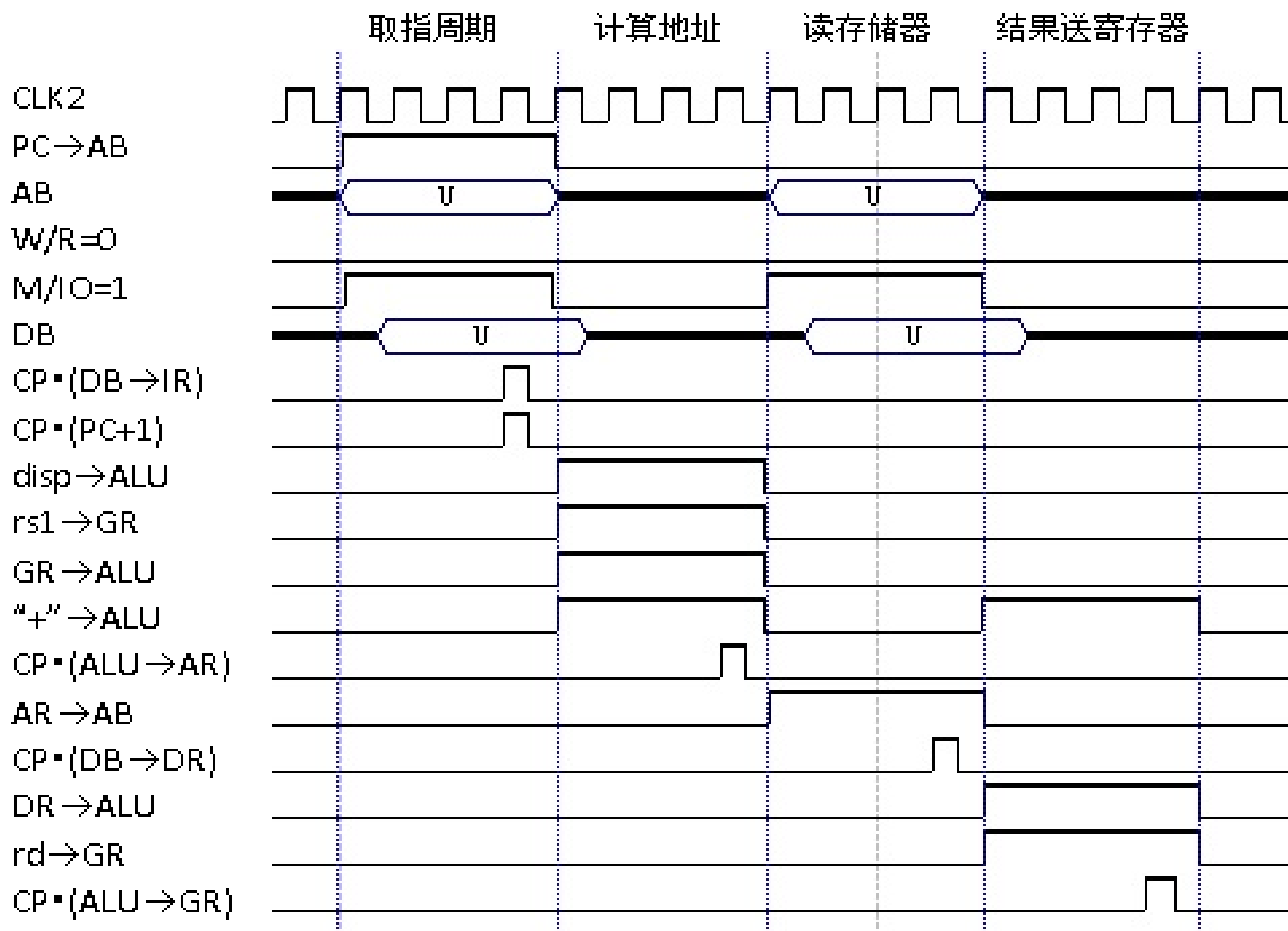
- 地址的运算结果保存在AR中，根据AR中的地址，读取操作数送到数据寄存器DR。
- CPU发出的控制信号如下：
 - $AR \rightarrow AB$ ：将寄存器AR中的地址送地址总线AB
 - $W/R=0$ ：发读控制信号给存储器，存储器读
 - $M/IO=1$ ：选择存储器操作
 - $CP \cdot (DB \rightarrow DR)$ ：将数据总线上的数据（存储器的读出数据）送入数据寄存器DR中

LOAD指令的分析

4. 结果回送：把存储单元的数据送到寄存器rd。

- 读出的存储单元数据在寄存器DR中，需要占用一个机器周期将其回送到寄存器rd中。
- DR没有直接通往GR的路径，必须利用ALU的传输途径，即ALU做一次加法，其中一个输入为DR寄存器，另一个输入控制信号不发出，输入值默认为0。
- 最后，将ALU的运算结果送入GR的寄存器rd中。
- CPU发出的控制信号如下：
 - DR→ALU，DR寄存器的数据输出到ALU
 - “+” →ALU，ALU选择加法操作
 - rd→GR，目的寄存器地址rd送GR
 - CP•(ALU→GR)，ALU运算结果送GR的rd中。

LOAD 指令的工作时序图



2) ADD指令分析

- add完成寄存器之间的加法操作，格式如下：

add rs/rd, rs1

- 表示将寄存器rs/rd的内容和寄存器rs1的内容相加，结果送到寄存器rs/rd中
- add指令一共占用2个机器周期
 - 取值令占用一个机器周期
 - 使用ALU计算占用一个机器周期

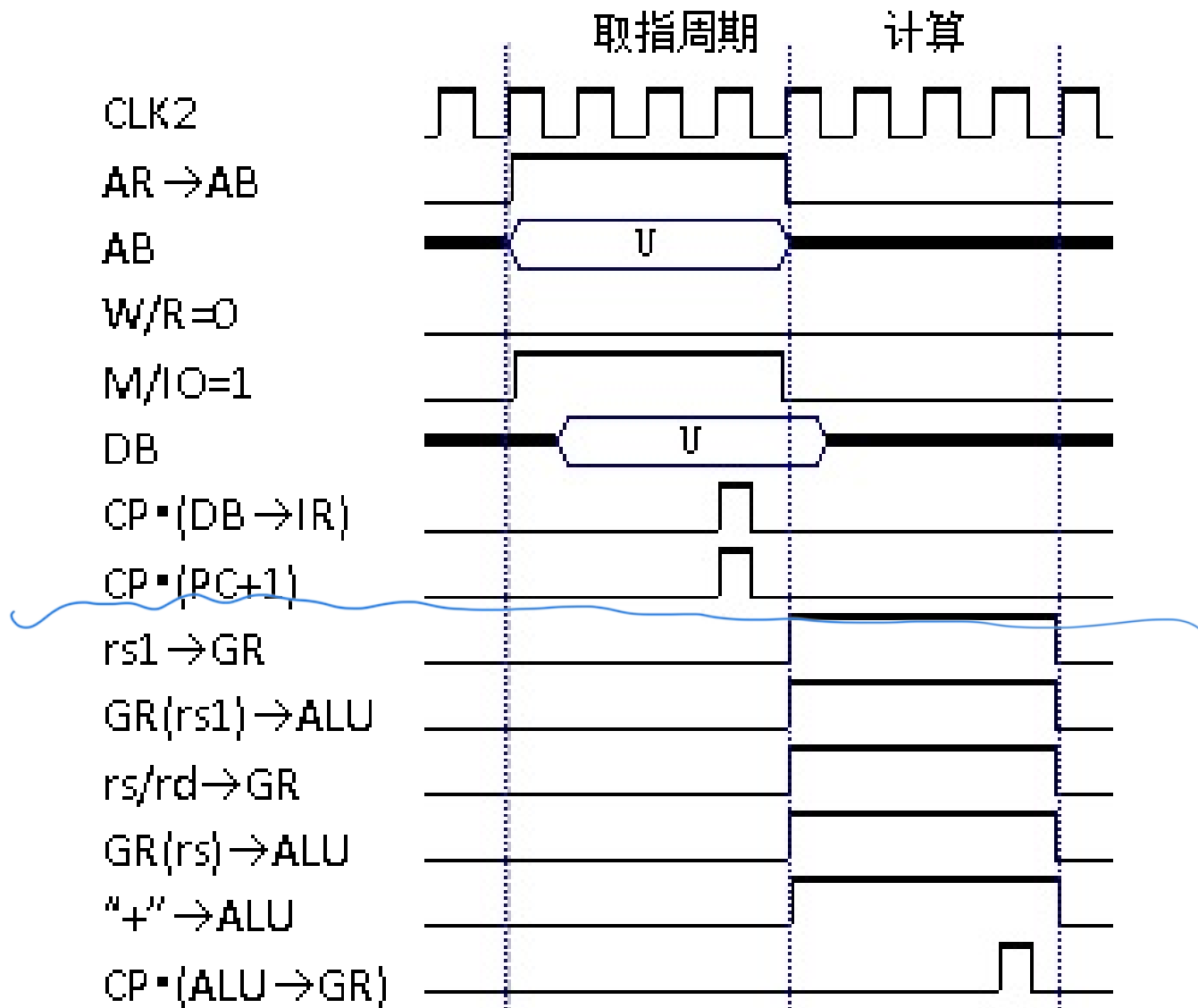
ADD指令的分析

1. **取指**：操作内容同公共机器周期。

2. **加法计算**：通过ALU实现加法运算，

- 2个源操作数都在GR内，1个目的地址也在GR内。
- 完成加法操作需要向GR发出2个寄存器地址信息，寄存器rs1输出一路数据到ALU左输入端，寄存器 rs输出一路数据到ALU右输入端。
- 向ALU发出操作选择信号 “+” ；经过一段延迟，计算结果将出现在ALU的输出端，发出控制信号让ALU的输出数据进入GR。
- CPU发出的控制信号如下：
 - rs/rd→GR，寄存器源/目的地址送GR
 - GR(rs/rd)→ALU，寄存器数据输出到ALU右端
 - rs1→GR，寄存器源地址送GR
 - GR(rs1)→ALU，寄存器数据输出到ALU左端
 - “+” →ALU，ALU选择加法操作
 - CP•(ALU→GR)，ALU计算结果送GR的rs/rd中。

ADD指令工作时序



3) BRANCH指令分析

- branch指令: branch cond disp

- 表示当条件cond满足时, 程序转向执行PC+disp所指向地址, 否则顺序执行, 即执行指令的下一条指令。

- 本指令占用机器周期的数目事先不能确定, 根据执行指令时cond的条件, 可能是一个机器周期或两个机器周期。

- 指令的第一个机器周期是取指周期, 内容同公共机器周期。
- 取指后, 先译码, 译码后计算条件转移状态, 采用硬件实现条件判断。
 - 如果条件不满足, 本机器周期结束之后指令执行结束, 下一步进入新的取指周期
 - 如果条件满足, 下一个机器周期进入执行周期, 计算PC的偏移量之和, 完成PC+disp→PC。

BRANCH指令的分析

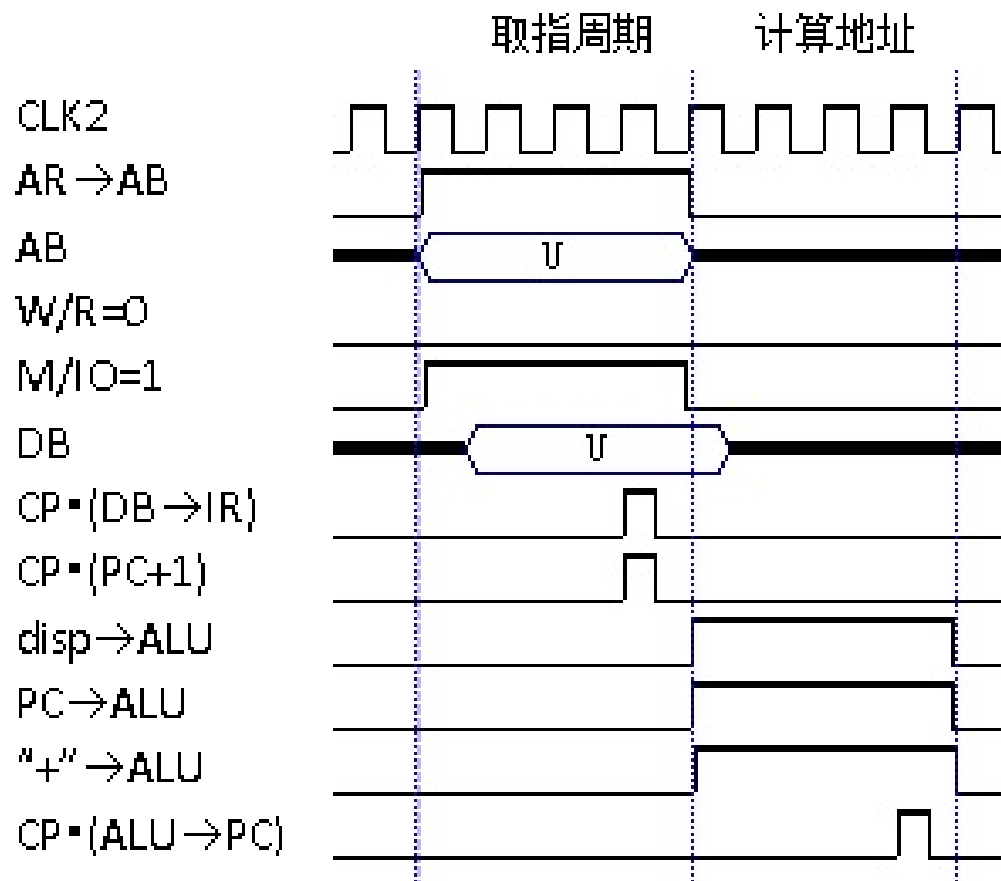
1. **取指**： 操作内容同公共机器周期。

2. **计算相对转移地址（cond成立时执行）**： 实现对PC值的修改， 经过ALU的加法运算。PC和disp分别送ALU的两个输入端。

■ CPU发送的控制信号如下：

- PC→ALU， 寄存器PC的数据送ALU的左输入端
- disp→ALU， 地址偏离量送ALU的右输入端
- “+” →ALU， ALU选择加法操作
- CP•(ALU→PC)， ALU运算结果送寄存器PC

BRANCH指令工作时序



4.2.3 CPU控制器实现技术

- 控制信号产生部件

- 以时钟、工作周期信号、指令译码器的输出，和指令执行过程的一些状态信号作为输入，产生控制指令执行的所有控制信号的部件

- 常见实现方法

- ① 微程序控制方法：存
 - ② 硬布线逻辑电路方法：算

① 微程序控制方法

■ 微程序控制器的基本思想

- ① 仿照程序设计的方法，编制每条指令所对应的微程序。
- ② 每个微程序由若干条微指令构成，一条微指令包含若干位微命令控制微操作。



一条微指令相当于一个状态，微命令就是状态中的控制信号

- ③ 所有微程序放在控制存储器(Control Memory)中，执行某条指令时，取出对应微程序中的各条微指令，对微指令译码产生对应的微命令，这个微命令就是控制信号。

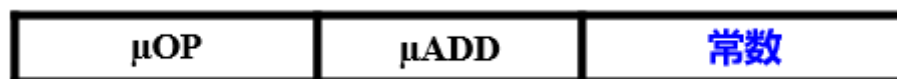
■ 微程序控制器的特点

- 具有规整性、可维性和灵活性，但速度慢

控制存储器

■控制存储器：简称控存

- 存放实现全部指令系统的所有微程序，是一种只读存储器
- 每一行对应一条微指令
- 微指令中包含若干微命令（对应所有控制信号）、下条微指令地址(可选)、常数(可选)

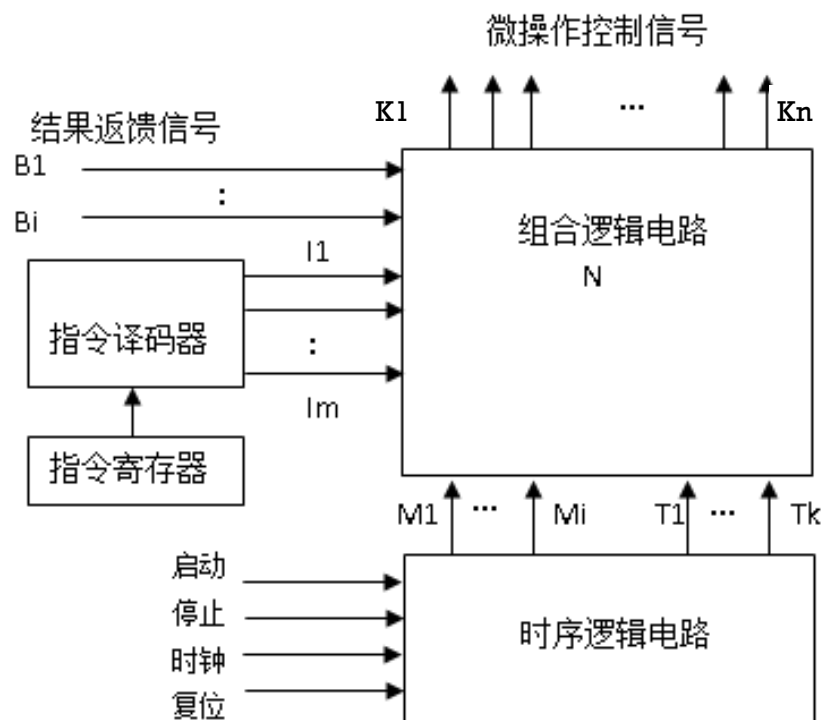


- μ OP：微操作码字段，产生微命令
- μ ADD：微地址码字段，产生下条微指令地址
- 微指令格式设计风格取决于微操作码的编码方式

② 硬布线逻辑电路方法

■ 硬布线控制器:又称组合逻辑控制器

- 主要由组合逻辑电路、指令寄存器和指令译码器、节拍电位/节拍脉冲发生器等部分组成
- 其中组合逻辑电路产生计算机所需的全部操作命令，是控制器的核心。



硬布线逻辑电路方法

■ **输入信号:** 硬布线逻辑控制器通过对一系列输入信号的逻辑运算得到一系列输出控制信号, 涉及到的输入信号如下:

- ① 来自**指令操作码译码器**的输出 $I_1 \sim I_m$, 译码器每根输出线表示一条指令, 译码器的输出反映出当前正在执行的指令;
- ② 来自执行部件的**反馈信息** $B_1 \sim B_j$;
- ③ 来自时序产生器的**时序信号**, 包括节拍电位信号 $M_1 \sim M_i$ 和节拍脉冲信号 $T_1 \sim T_k$ 。其中节拍电位信号就是机器周期(CPU周期)信号, 节拍脉冲信号是时钟周期信号。

■ 输出信号

- 组合逻辑电路的输出信号就是微操作控制信号 $K_1 \sim K_n$, 用来对执行部件进行控制。
- 另有一些信号则根据条件变量来改变时序发生器的计数顺序, 以便跳过某些状态, 从而可以缩短指令周期。

硬布线逻辑电路方法

- 硬布线控制器设计的基本原理：某一微操作控制信号 K_i 是指令操作码译码器输出 I_m 、时序信号(节拍电位 M_i ，节拍脉冲 T_k)和状态条件信号 B_j 的逻辑函数，

$$K_i = f(I_m, M_i, T_k, B_j)$$

- 微操作控制信号 K_i ：在某条特定指令和状态条件下，在某一操作的特定节拍电位和节拍脉冲时间间隔中起作用，从而激活这条控制信号线，对执行部件实施控制。
- 例如，对引起一次主存读操作的控制信号 K_3 来说，

- 当节拍电位 $M_1=1$ ，取指令时被激活；

- 而节拍电位 $M_3=1$ ，三条指令（Load，add，and）取操作数时也被激活，此时指令译码器的Load，add，and输出均为1，因此 K_3 的逻辑表达式可由下式确定：

$$K_3 = M_1 + \overbrace{M_3 (\text{Load} + \text{add} + \text{and})}^{\text{取指+计算+取数}}$$

处于 M_3 机器周期，有内存读操作

硬布线逻辑设计过程

- ① 采用适宜的指令格式，合理分配指令操作码；
- ② 确定机器周期、节拍与主频；
CPU给定 *一条指令几个周期？*
每个周期干什么事？
- ③ 确定机器周期数及一周期内的操作；
- ④ 进行指令综合：综合所有指令的每一个操作命令，写出逻辑表达式，并进行化简。
- ⑤ 明确组合逻辑电路：将简化后的逻辑表达式用组合逻辑电路来实现。
 - 整个CPU的指令很多条，设计过程中，需要针对每条指令进行详细的分析和设计。
 - 设计每个控制信号的产生逻辑：针对各个控制信号，需要确定其逻辑表达式。将所有的指令的每个机器周期的微操作都设计出来后，就可以得到每个控制信号

硬布线逻辑设计举例

假如CPU具有指令load、store、add、sub、branch,...等众多的指令，现选择几个典型指令：

- load和store指令：占4个机器周期 (C_1 、 C_2 、 C_3 、 C_4)
-  add和sub指令：占2个机器周期 (C_1 、 C_2)
- branch指令：取出指令之后，还不能确定一共需要多少机器周期，需要硬件逻辑线路对指令给出的条件进行判定。
 - 如果条件不成立，则需要1个机器周期，下个机器周期为新指令的取指周期；
 - 如果条件成立，则需要2个机器周期，下个机器周期为计算地址周期。
 - 因此，branch指令可能占一个机器周期 C_1 ，也可能占 C_1 、 C_2 个机器周期。

硬布线逻辑设计举例

- 机器周期通过控制信号 C_1 、 C_2 、 C_3 、 C_4 表示4个机器周期。

- ^{取指}
~~取值~~周期 C_1 是每条指令必须执行的，因此，公共机器周期 C_1 中，控制信号的产生逻辑如下：

$$PC \rightarrow AB = C_1$$

$$W/\overline{R} = \overline{C_1} \quad \text{w/r 表示读给0, 写给1}$$

$$M/\overline{IO} = C_1 \quad \text{内存. 给1}$$

$$DB \rightarrow IR = C_1$$

$$PC + 1 = C_1 \cdot CP$$

硬布线逻辑设计举例

读写信号 $W/\overline{R}$

- 在 C_1 取指周期，取值令执行读操作， $W/\overline{R}=0$ ，所以 $W/\overline{R}=\overline{C_1}$;
- 在 C_3 取数周期，load指令执行读操作， $W/\overline{R}=0$ ，所以 $W/\overline{R}=\overline{C_3}$;
- 在 C_4 存数周期，store指令执行写操作， $W/\overline{R}=1$ ，所以 $W/\overline{R}=C_4$;

因此， $W/\overline{R}$ 的逻辑函数如下：

$$W/\overline{R} = \underbrace{\text{store}}_{\text{操作}} \cdot \underbrace{C_4}_{\text{周期数}} \quad (\text{默认值为0})$$

操作 周期数

其他情况，无信号，或为0

硬布线逻辑设计举例

加信号 “+” $\rightarrow$ ALU

◆在C₂周期，下面的指令需要使用ALU：

- 计算地址周期(load和store指令)，使用ALU计算地址；
- add指令使用ALU加法计算；
- 转移地址计算(branch转移成功等)，使用ALU计算地址；

◆ALU做加法的控制信号的产生逻辑函数如下：

$$\text{“+”} \rightarrow \text{ALU} = C_2 \cdot (\text{load} + \text{store} + \text{add} + \text{branch} \cdot (\text{条件成立}))$$

其他的控制信号以此类推，最后完成所有控制信号的逻辑函数的设计。

硬布线逻辑和微程序控制器的比较

1. 实现方式

由于硬布线逻辑一旦控制部件构成后，除非重新设计和物理上对它重新布线，否则要想增加新的控制功能是不可能的。

硬布线控制器是计算机中最复杂的逻辑部件之一。结构上的这种缺陷使得硬布线控制器的设计和调试非常复杂且代价很大。

2. 性能方面

同样半导体工艺条件下，微程序控制的速度比硬布线控制的速度低，因为执行每条微程序指令都要从控制存储器中读取，影响了速度。在超高速机器中，对影响速度的关键部分，往往采用硬布线逻辑实现。

随着新一代机器及VLSI技术的发展与不断进步，现代新型计算机体系结构如RISC中多采用硬布线控制逻辑。